

Tổng quan về xây dựng mô hình trên dữ liệu

Contents

- 3.1. Giới thiệu chung về xây dựng mô hình
- 3.2. Học máy và ngôn ngữ Python
- 3.3. Ứng dụng thực tế của mô hình học máy



KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

3.1. Giới thiệu chung về xây dựng mô hình

Học máy, còn gọi là **machine learning** là một lĩnh vực nghiên cứu liên ngành, nằm tại giao điểm của thống kê, trí tuệ nhân tạo và khoa học máy tính. Mục tiêu cốt lõi của học máy là **trích xuất tri thức từ dữ liệu** thông qua các thuật toán và mô hình toán học. Trong một số tài liệu, học máy còn được gọi là **học thống kê** (statistical learning) hoặc **phân tích dự báo** (predictive analytics).

Trong những năm gần đây, học máy đã trở thành một trong những công nghệ nền tảng được tích hợp rộng rãi trong các hệ thống và ứng dụng hiện đại. Các thuật toán học máy đang vận hành phía sau nhiều chức năng quen thuộc trong đời sống hàng ngày, từ việc đề xuất phim trên nền tảng giải trí, gợi ý sản phẩm trong thương mại điện tử, đến cá nhân hóa nội dung truyền thông và nhận diện khuôn mặt trong ảnh. Hầu hết các dịch vụ lớn như Facebook,

Amazon hoặc Netflix đều tích hợp nhiều mô hình học máy trong hầu hết các thành phần giao diện và chức năng cốt lõi.

Không chỉ giới hạn trong các ứng dụng thương mại, học máy còn có vai trò quan trọng trong nghiên cứu khoa học hiện đại. Các công cụ và thuật toán học máy đã được áp dụng để giải quyết nhiều vấn đề phức tạp như phân tích dữ liệu thiên văn, phát hiện hành tinh ngoài hệ Mặt Trời, khám phá hạt cơ bản trong vật lý năng lượng cao, giải mã trình tự DNA, và phát triển các liệu pháp điều trị ung thư cá thể hóa.

Tuy nhiên, để ứng dụng học máy hiệu quả, không nhất thiết phải xử lý các vấn đề ở quy mô toàn cầu hay có tác động lớn. Ngay cả những tác vụ nhỏ, ở phạm vi tổ chức hoặc cá nhân, cũng có thể hưởng lợi từ việc áp dụng các kỹ thuật học máy một cách hợp lý.

Trong chương này, chúng ta sẽ cùng khám phá lý do tại sao học máy ngày càng phổ biến, và phân tích các loại bài toán mà học máy có thể giải quyết hiệu quả. Đồng thời, chương trình học sẽ hướng dẫn người đọc xây dựng mô hình học máy đầu tiên, qua đó giới thiệu các khái niệm cốt lõi và phương pháp tiếp cận trong thực tiễn.

Trước khi học máy trở nên phổ biến, nhiều hệ thống được thiết kế theo phương pháp **mã hóa thủ công** (rule-based systems), trong đó các quyết định được đưa ra dựa trên các cấu trúc điều kiện như "if-else". Ví dụ, một bộ lọc thư rác có thể được xây dựng bằng cách liệt kê các từ khóa nghi ngờ để đánh dấu email là thư rác. Đây là cách tiếp cận sử dụng tri thức chuyên gia để xây dựng một hệ thống ra quyết định.

Tuy nhiên, cách tiếp cận này tồn tại hai hạn chế cơ bản:

- Logic ra quyết định thường chỉ áp dụng cho một lĩnh vực hoặc nhiệm vụ cụ thể. Việc thay đổi yêu cầu hoặc áp dụng sang bối cảnh khác thường đòi hỏi phải thiết kế lại toàn bộ hệ thống.
- Việc thiết kế các quy tắc phụ thuộc nặng nề vào tri thức chuyên gia, và không phải lúc nào con người cũng có khả năng mô hình hóa rõ ràng quá trình nhận thức hoặc ra quyết định.

Một ví dụ điển hình cho hạn chế của phương pháp dựa trên quy tắc là trong bài toán phát hiện khuôn mặt trong ảnh số. Mặc dù ngày nay hầu hết các thiết bị thông minh đều có khả năng nhận diện khuôn mặt, nhưng đây từng là một bài toán khó chưa được giải quyết hiệu quả cho đến năm 2001. Nguyên nhân là do cách máy tính biểu diễn hình ảnh (dưới dạng ma trận các điểm ảnh) rất khác với cách con người nhận thức khuôn mặt. Điều này khiến việc mô tả khuôn mặt bằng các quy tắc cố định trở nên cực kỳ khó khăn và thiếu khả năng tổng quát.

Ngược lại, học máy cho phép hệ thống tự động học các mẫu và đặc điểm từ dữ liệu đầu vào. Trong bài toán nhận diện khuôn mặt, thay vì liệt kê các đặc điểm thủ công, ta chỉ cần cung cấp một tập hợp lớn các hình ảnh có chứa khuôn mặt. Thuật toán học máy sẽ tự động phát hiện các đặc trưng phân biệt khuôn mặt với các đối tượng khác, từ đó xây dựng mô hình nhận dạng hiệu quả.

Khả năng học từ dữ liệu giúp học máy linh hoạt, thích ứng với nhiều loại bài toán khác nhau mà không đòi hỏi hiểu biết tường minh về tất cả các quy luật hoặc mối quan hệ bên trong dữ liệu. Chính vì thế, học máy ngày càng trở thành công cụ quan trọng trong nhiều ngành nghề, từ công nghiệp, y tế, tài chính đến khoa học xã hội và giáo dục.

3.1.1. Lịch sử hình thành của các mô hình học máy

Mặc dù thuật ngữ **mô hình học máy** còn khá mới mẻ, nhưng những kỹ thuật nền tảng cho lĩnh vực này đã được phát triển từ lâu.

Vào đầu thế kỷ 19, phương pháp bình phương nhỏ nhất đã được phát triển và áp dụng để ước lượng các mô hình **hồi quy tuyến tính**. Mô hình này lần đầu tiên được áp dụng và cho kết quả thành công trong các vấn đề liên quan đến thiên văn học.

Vào đầu thế kỷ 20, mô hình hồi quy tuyến tính được sử dụng để dự đoán các giá trị định lượng, chẳng hạn như mức lương của một cá nhân hoặc để dự đoán các giá trị định tính, chẳng hạn như bệnh nhân sống hay chết, hay thị trường chứng khoán tăng hay giảm.

Vào những năm 1940, nhiều tác giả đã đưa ra một cách tiếp cận khác, đó là **hồi quy logistic**. Vào đầu những năm 1970, thuật ngữ mô hình **tuyến tính tổng quát** đã được phát triển để mô tả toàn bộ lớp phương pháp học thống kê bao gồm cả hồi quy tuyến tính và hồi quy logistic như các trường hợp đặc biệt.

Vào cuối những năm 1970, nhiều kỹ thuật xây dựng mô hình trên dữ liệu đã xuất hiện. Tuy nhiên, các mô hình này chỉ xoay quanh các phương pháp tuyến tính vì việc tạo ra các mối quan hệ phi tuyến tính rất khó khăn về mặt tính toán.

Đến những năm 1980, sự phát triển của máy tính điện tử đã hỗ trợ tích cực về mặt tính toán cho các phương pháp phi tuyến tính. Các mô hình phi tuyến được giới thiệu vào đầu những năm 1980 bao gồm mô hình **cây quyết định** và mô hình **cộng tính tổng quát**. Những năm cuối thập niên 1980 và đầu thập niên 1990, mô hình **mạng nơ-ron** được giới thiệu đến cộng

đồng nghiên cứu nhưng chưa nhận được nhiều sự quan tâm vì dữ liệu chưa đủ phong phú và sự phổ biến của các mô hình học máy khác.

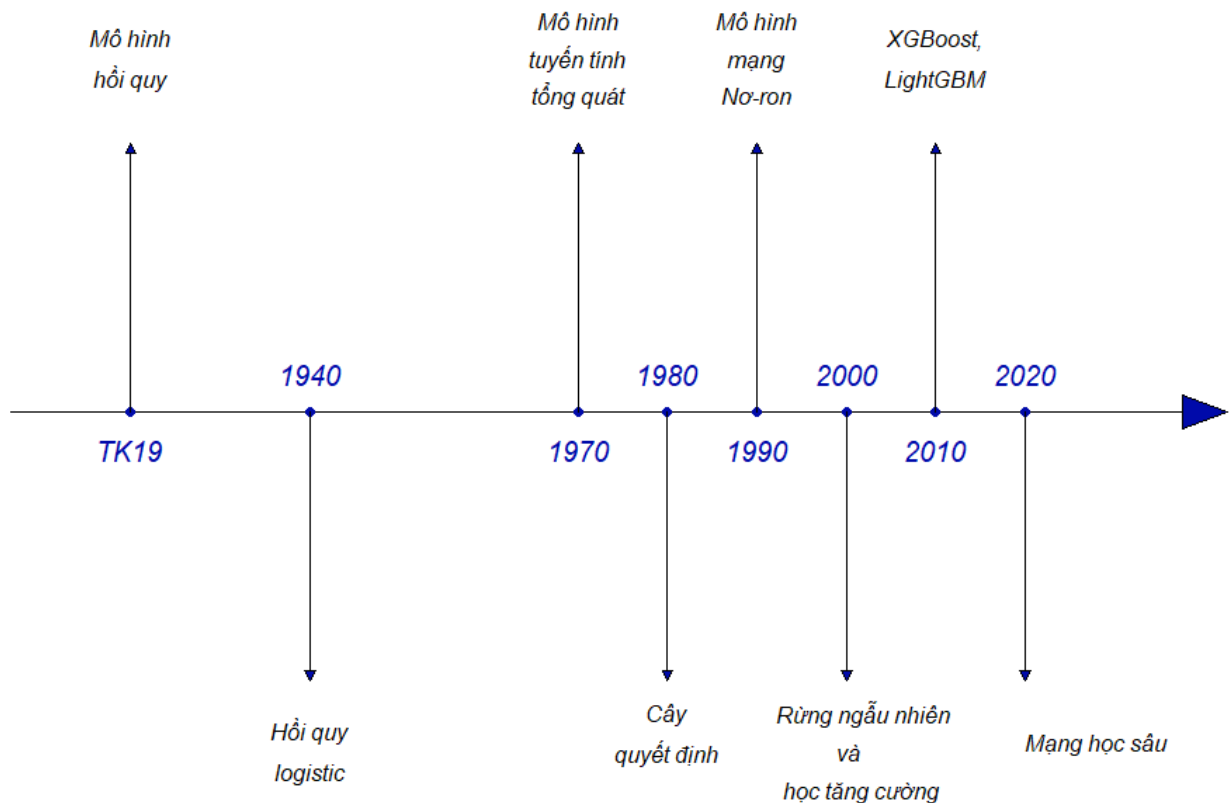


Fig. 3.1 Quá trình phát triển của các mô hình học máy

Giai đoạn cuối thế kỷ XX và đầu thế kỷ XXI là giai đoạn chiếm ưu thế hoàn toàn của các mô hình học máy **rừng ngẫu nhiên** và thuật toán **học tăng cường**. Thuật toán học tăng cường với các biến thể như **XGBoost** hay **LightGBM** chiến thắng trong hầu hết các cuộc thi về khoa học dữ liệu.

Từ năm 2010, với sự bùng nổ của các thiết bị thông minh và kết nối internet, dữ liệu trở nên phong phú và đa dạng hơn cũng là thời điểm quay trở lại của mô hình mạng nơ-ron, hay còn được gọi với tên gọi khác là mô hình **mạng học sâu** (deep learning). Mô hình mạng học sâu vượt trội hoàn toàn các mô hình học máy thông thường khi làm việc với dữ liệu kiểu hình ảnh, video, ngôn ngữ tự nhiên bao gồm cả văn bản và giọng nói.

Sự kiện đánh dấu sự phát triển vượt bậc của các mô hình mạng học sâu là sự ra đời của ChatGPT vào cuối năm 2022, một mô hình **ngôn ngữ lớn** cho phép người dùng tương tác, hỏi đáp và trò chuyện một cách hoàn toàn tự nhiên theo định hướng của người sử dụng như phong cách, mức độ chi tiết, hình thức ngôn ngữ.

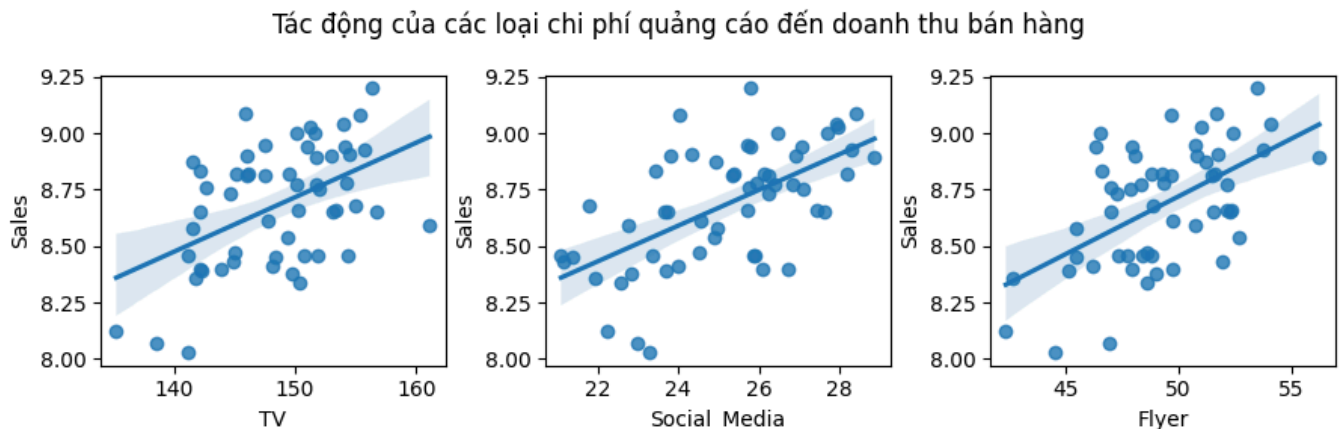
ChatGPT nhanh chóng đạt đến 100 triệu người dùng sau hơn hai tháng phát hành và giúp cho công ty phát hành OpenAI được định giá khoảng 30 tỷ USD. Cho đến thời điểm cuối năm 2024 khi nhóm tác giả bắt đầu viết cuốn sách này, ChatGPT đã được cập nhật đến phiên bản 4.0.

3.1.2. Học máy trong kinh tế và kinh doanh?

Việc sử dụng dữ liệu và các mô hình học máy để hỗ trợ ra quyết định trong kinh tế và kinh doanh, thay vì chỉ dựa trên kinh nghiệm thuần túy, không chỉ giúp các nhà quản lý có cái nhìn toàn diện hơn về tình hình kinh doanh mà còn giúp họ đưa ra những quyết định khách quan, thông minh và hiệu quả hơn.

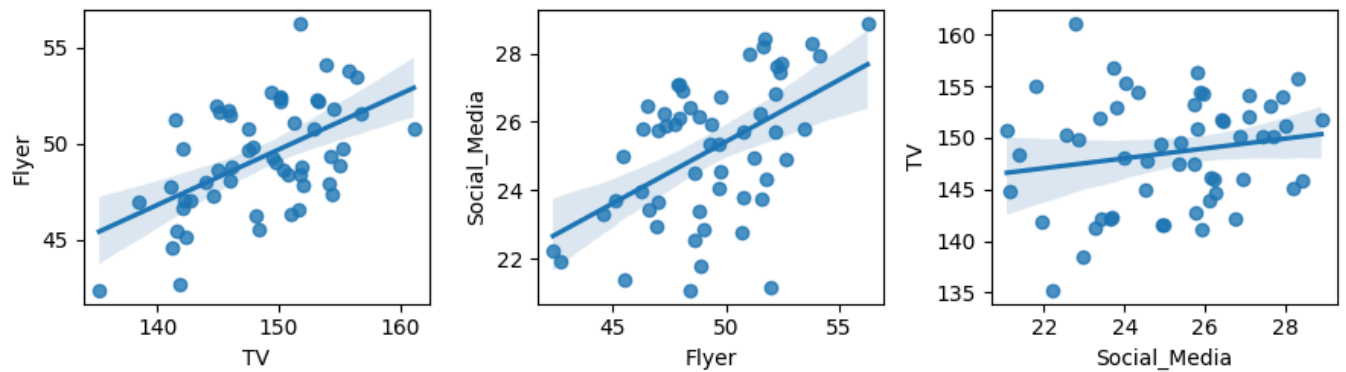
Để minh họa ứng dụng của xây dựng mô hình trên dữ liệu trong lĩnh vực kinh tế và kinh doanh, chúng tôi thảo luận ngắn gọn về dữ liệu từ một chiến dịch quảng cáo của một công ty thu thập được từ 55 cửa hàng trên toàn quốc. Dữ liệu bao gồm thông tin về doanh thu bán sản phẩm và chi phí công ty đã chi cho ba phương thức quảng cáo là quảng cáo qua truyền hình, quảng cáo qua các nền tảng mạng xã hội, và quảng cáo bằng hình thức phát tờ rơi. Mỗi liên hệ của chi phí cho mỗi phương thức quảng cáo đến doanh thu từ bán sản phẩm được mô tả trong hình

► Show code cell source



► Show code cell source

Tác động của qua lại của các loại chi phí quảng cáo



Từ mối liên hệ của các biến có thể đưa ra nhận định khá chắc chắn là chi tiền cho quảng cáo có ý nghĩa trong việc tăng doanh thu bán sản phẩm. Có thể thấy trong các đồ thị ở hàng phía trên rằng có mối liên hệ cùng chiều giữa chi phí chi cho các hình thức quảng cáo đến doanh thu bán sản phẩm. Mối liên hệ này là phù hợp với logic nói chung về quảng cáo sản phẩm: khi công ty chi tiền cho quảng cáo, nhiều khách hàng sẽ có cơ hội tiếp cận về sản phẩm hơn, làm tăng số lượng người mua sản phẩm và tăng doanh thu cho các cửa hàng.

Tuy nhiên, vẫn còn rất nhiều câu hỏi cần giải đáp từ đối với các nhà quản lý:

1. Mối liên hệ giữa ngân sách chi cho quảng cáo và doanh thu nếu tồn tại thì mạnh đến mức nào?
2. Phương tiện truyền thông nào góp phần tăng doanh thu bán hàng? Cả ba phương tiện truyền thông qua Tivi, mạng xã hội và phát tờ rơi có đóng góp vào doanh thu bán hàng hay chỉ một hoặc hai phương tiện quảng cáo có đóng góp?
3. Chúng ta có thể ước tính chính xác tác động của chi phí từng phương tiện quảng cáo đến doanh thu bán hàng như thế nào? Với cùng một mức chi cho quảng cáo trên một phương tiện cụ thể, doanh thu bán hàng sẽ tăng bao nhiêu? Chúng ta có thể dự đoán mức tăng này chính xác đến mức nào?
4. Mối liên hệ hay sự tác động của chi phí cho từng hình thức quảng cáo đến doanh thu bán hàng có tuyến tính không? Nếu không, liệu có phương pháp biến đổi như thế nào để mối liên hệ vẫn là tuyến tính?
5. Có sự tác động qua lại giữa các chi phí cho các phương tiện quảng cáo không? Chẳng hạn như nên chi đồng thời 10 triệu đồng cho quảng cáo trên mạng xã hội và 10 triệu đồng cho quảng cáo tờ rơi liệu có mang lại doanh thu cao hơn việc phân bổ 20 triệu cho riêng từng hình thức?

Để trả lời các câu hỏi kể trên, nhà quản lý cần xây dựng một mô hình phản ánh tác động của các chi phí cho quảng cáo lên doanh thu bán hàng. Về bản chất, mô hình cần xây dựng là một

hàm số mô tả mối liên hệ giữa chi phí quảng cáo và doanh thu bán sản phẩm, tạm gọi là hàm f như sau:

$$\text{Sales} = f(\text{TV}, \text{SocialMedia}, \text{Flyer}) + \epsilon \quad (3.1)$$

với ϵ là sai số trong mô hình mà không thể giải thích được bằng dữ liệu. Nói cách khác, ϵ đại diện cho ảnh hưởng của các yếu tố có thể có tác động đến doanh thu nhưng không liên quan đến 3 loại hình quảng cáo trên.

Trong xuyên suốt cuốn sách, nếu không có giả thiết bổ sung, chúng tôi luôn mặc định là sai số có trung bình bằng 0 và phương sai là một hằng số.

$$\begin{aligned} \mathbb{E}(\epsilon) &= 0 \\ \mathbb{V}(\epsilon) &= \sigma^2 \end{aligned}$$

Trong (3.1), biến **Sales** được gọi là biến phụ thuộc, biến phụ thuộc, hoặc biến đầu ra, trong khi các biến **TV**, **SocialMedia**, và **Flyer** được gọi là các biến giải thích, biến độc lập, hoặc biến đầu vào. Trong cuốn sách này chúng tôi sẽ sử dụng cặp khái niệm biến phụ thuộc - biến giải thích và biến phụ thuộc - biến độc lập thay thế cho nhau tùy vào từng ngữ cảnh của mô hình.

Khi thảo luận về các mô hình và không đề cập đến một dữ liệu cụ thể, chúng tôi sẽ sử dụng ký hiệu Y để mô tả biến phụ thuộc và $\mathbf{X} = (X_1, X_2, \dots, X_p)$ để mô tả véc-tơ các biến độc lập. Một cách tổng quát, chúng ta cần xác định mô hình trên dữ liệu, được gọi tắt là f , sao cho

$$Y = f(\mathbf{X}) + \epsilon$$

với ϵ là sai số độc lập với biến giải thích $\mathbf{X}$.

Hàm f trong công thức trên là một hàm số xác định nhưng chưa biết. Mục tiêu quan trọng nhất của những người xây dựng mô hình là ước lượng hàm f . Tuy nhiên, tất cả thông tin mà người xây dựng mô hình có được chỉ là các giá trị quan sát được của các biến độc lập và biến phụ thuộc tương ứng, mà không có bất kỳ gợi ý nào về dạng của hàm f . Một ước lượng hàm f , thường được ký hiệu là $\hat{f}$, có phải là một xấp xỉ tốt cho hàm f hay không hoàn toàn phụ thuộc vào kinh nghiệm và kỹ năng của người xây dựng mô hình.

Trong phần tiếp theo của cuốn sách chúng tôi sẽ thảo luận về những cân nhắc mà người xây dựng mô hình cần phải đặt ra trước khi tiến hành ước lượng hàm f .

3.1.3. Các vấn đề học máy có thể giải quyết

Một trong những nhóm thuật toán học máy thành công nhất là các thuật toán có khả năng **tự động hóa quá trình ra quyết định** bằng cách **tổng quát hóa từ các ví dụ đã biết**. Trong bối cảnh này, phương pháp được sử dụng thường được gọi là **học có giám sát**, hay *supervised learning*, trong đó người dùng cung cấp cho thuật toán một tập hợp các cặp đầu vào và đầu ra mong muốn. Mục tiêu của thuật toán là học cách ánh xạ từ đầu vào tới đầu ra, và từ đó có khả năng dự đoán đầu ra đối với những đầu vào chưa từng thấy trước đó.

Chẳng hạn, trong bài toán phân loại thư rác, một thuật toán học có giám sát sẽ được huấn luyện trên một tập hợp các email đầu vào, cùng với nhãn tương ứng cho biết đó có phải là thư rác hay không. Sau quá trình học từ dữ liệu, thuật toán có thể dự đoán tính chất của một email mới mà không cần sự can thiệp trực tiếp của con người.

Thuật ngữ “có giám sát” bắt nguồn từ việc mỗi ví dụ trong dữ liệu huấn luyện được gắn với một nhãn rõ ràng, đóng vai trò như một người giám sát cung cấp thông tin phản hồi trong quá trình mô hình học từ dữ liệu. Mặc dù việc xây dựng một tập dữ liệu có nhãn thường là một công việc thủ công và tốn kém, nhưng các thuật toán học có giám sát hiện nay đã được nghiên cứu kỹ lưỡng và hiệu suất của chúng có thể được đánh giá một cách rõ ràng bằng các tiêu chí định lượng.

Nếu bài toán của bạn có thể được biểu diễn dưới dạng một ánh xạ từ đầu vào đến đầu ra và bạn có thể thu thập đủ dữ liệu có nhãn, thì học có giám sát là một công cụ tiềm năng để giải quyết vấn đề đó. Một số ví dụ về học có giám sát là:

- *Dự đoán giá bất động sản*: đầu vào là các thông tin chi tiết về một bất động sản cụ thể, chẳng hạn như diện tích, số phòng ngủ, chiều rộng mặt tiền, vị trí, loại hình nhà ở, năm xây dựng, v.v.; với đầu ra là một giá trị liên tục thể hiện giá nhà dự kiến (ví dụ: triệu đồng/m²). Tập dữ liệu được xây dựng dựa trên các bất động sản đã được giao dịch thành công trong quá khứ, với thông tin đầy đủ về đặc trưng và mức giá thực tế tại thời điểm bán.
- *Dự đoán khả năng chấp nhận dịch vụ của khách hàng*: đầu vào là các thông tin liên quan đến hồ sơ và hành vi của khách hàng, chẳng hạn như độ tuổi, giới tính, thu nhập, lịch sử tương tác với doanh nghiệp, số lần nhận email quảng cáo, thời điểm tương tác gần nhất, v.v.; với đầu ra là nhãn “chấp nhận” hoặc “không chấp nhận”. Tập dữ liệu được xây dựng từ các chiến dịch marketing trong quá khứ, nơi khách hàng đã được tiếp cận với dịch vụ và ghi nhận phản hồi thực tế (ví dụ: đăng ký sử dụng hoặc từ chối). Đây là một bài toán phân

loại nhị phân thường được áp dụng trong lĩnh vực ngân hàng, bảo hiểm, viễn thông và thương mại điện tử.

- *Phát hiện giao dịch gian lận trong thẻ tín dụng*: đầu vào là thông tin về một giao dịch cụ thể, chẳng hạn như giờ giao dịch, địa điểm, số tiền, v.v.; với đầu ra là "gian lận" hoặc "không gian lận". Tập dữ liệu được xây dựng dựa trên các giao dịch lịch sử mà khách hàng đã xác nhận là hợp lệ hoặc không hợp lệ.
- *Xác định mã zip từ chữ viết tay trên phong bì*: đầu vào là ảnh quét của phần mã zip viết tay; đầu ra là dãy số trong mã zip. Để xây dựng tập dữ liệu huấn luyện, cần thu thập nhiều phong bì, trích xuất hình ảnh và gán nhãn bằng cách đọc mã zip tương ứng.
- *Phân loại khối u lành tính dựa trên ảnh y tế*: đầu vào là hình ảnh chẩn đoán y tế (ví dụ ảnh MRI); đầu ra là nhãn "lành tính" hoặc "ác tính". Tập dữ liệu yêu cầu hình ảnh y tế đi kèm với chẩn đoán chính xác từ bác sĩ chuyên khoa, đôi khi cần thêm các kiểm tra lâm sàng hỗ trợ.

Một điều đáng lưu ý là tuy các bài toán có cấu trúc đầu vào và đầu ra tương đối đơn giản, nhưng quá trình thu thập dữ liệu cho mỗi bài toán lại khác biệt rất lớn về chi phí, yêu cầu kỹ thuật, tính riêng tư và đạo đức.

Trái ngược với học có giám sát, **học không giám sát**, hay *unsupervised learning* là phương pháp mà trong đó chỉ có dữ liệu đầu vào, và **không có đầu ra mong muốn** được cung cấp. Mục tiêu của học không giám sát là khám phá các cấu trúc ẩn trong dữ liệu mà không cần đến nhãn. Tuy học không giám sát thường khó đánh giá hơn do thiếu đầu ra để so sánh, phương pháp này vẫn đóng vai trò quan trọng trong nhiều ứng dụng phân tích dữ liệu khám phá. Một số ví dụ về học không giám sát có thể kể đến:

- *Phân tích chủ đề từ tập hợp các bài viết và tin tức*: cho một tập lớn văn bản, mục tiêu là xác định các chủ đề phổ biến mà không cần biết trước các chủ đề là gì.
- *Phân khúc khách hàng theo hành vi tiêu dùng*: với dữ liệu hành vi khách hàng (lịch sử mua sắm, loại sản phẩm, thời điểm truy cập...), thuật toán sẽ phân nhóm khách hàng thành các cụm có đặc điểm tương đồng, ví dụ như "người mua theo mùa", "người thích công nghệ", "phụ huynh", v.v.
- *Phát hiện truy cập bất thường vào hệ thống*: các truy cập không thường xuyên, quá nhanh hoặc từ các địa chỉ IP đáng ngờ có thể được phát hiện bằng cách tìm ra các điểm dữ liệu nằm xa khỏi "cụm" bình thường trong không gian đặc trưng.

3.2. Học máy và ngôn ngữ Python

Python đã nổi lên như một ngôn ngữ lập trình phổ quát, đóng vai trò trung tâm trong nhiều ứng dụng thuộc lĩnh vực khoa học dữ liệu. Ngôn ngữ này kết hợp hiệu quả giữa sức mạnh của một ngôn ngữ lập trình đa năng với tính đơn giản và dễ tiếp cận của các ngôn ngữ chuyên biệt như MATLAB hoặc R. Python cung cấp một hệ sinh thái thư viện phong phú phục vụ cho hầu hết các tác vụ phân tích dữ liệu, bao gồm tải và xử lý dữ liệu, trực quan hóa, thống kê, học máy, xử lý ngôn ngữ tự nhiên, xử lý ảnh và nhiều lĩnh vực chuyên sâu khác.

Kho công cụ đa dạng và phát triển mạnh mẽ này đem đến cho các nhà khoa học dữ liệu một nền tảng lập trình linh hoạt, có thể đáp ứng cả các yêu cầu phân tích tổng quát lẫn các bài toán chuyên biệt. Một trong những điểm nổi bật của Python là khả năng hỗ trợ lập trình tương tác – cho phép người dùng viết, thực thi và kiểm tra mã theo từng bước trong môi trường như terminal hoặc Jupyter Notebook.

Trong bối cảnh học máy và phân tích dữ liệu hiện đại, việc lặp lại quá trình khám phá dữ liệu, tinh chỉnh mô hình và đánh giá kết quả là một yêu cầu thiết yếu. Python hỗ trợ tốt cho các quy trình lặp đi lặp lại này nhờ khả năng tương tác cao và thời gian phản hồi nhanh, từ đó thúc đẩy hiệu quả làm việc và sáng tạo của người phân tích.

Không dừng lại ở các ứng dụng phân tích, với tư cách là một ngôn ngữ lập trình tổng quát, Python còn cho phép phát triển các giao diện người dùng đồ họa (GUI), dịch vụ web, và tích hợp linh hoạt với các hệ thống phần mềm hiện có. Điều này giúp Python trở thành một công cụ không chỉ mạnh mẽ trong nghiên cứu mà còn có tính ứng dụng cao trong triển khai thực tế.

3.2.1. Thư viện xây dựng mô hình học máy

Scikit-learn là một thư viện học máy mã nguồn mở được phát triển cho ngôn ngữ lập trình Python. Việc phát hành dưới giấy phép mã nguồn mở không chỉ giúp người dùng sử dụng và phân phối tự do, mà còn cho phép bất kỳ ai có thể truy cập, đọc và đóng góp vào mã nguồn của dự án. Scikit-learn là một trong những thư viện học máy phổ biến và có ảnh hưởng nhất hiện nay trong cả môi trường học thuật và doanh nghiệp.

Thư viện này liên tục được duy trì và cải tiến bởi một cộng đồng phát triển năng động, với nhiều bản cập nhật được phát hành thường xuyên nhằm mở rộng tính năng, cải thiện hiệu suất và tăng cường độ ổn định. Scikit-learn cung cấp triển khai hiệu quả của nhiều thuật toán học máy cổ điển và hiện đại, bao gồm hồi quy tuyến tính và logistic, cây quyết định, k-láng giềng gần nhất, các phương pháp phân cụm, và nhiều kỹ thuật khác.

Một trong những điểm mạnh nổi bật của scikit-learn là hệ thống tài liệu trực tuyến đầy đủ và chi tiết. Mỗi thuật toán đều được trình bày rõ ràng với phần mô tả lý thuyết, giao diện lập trình, ví dụ sử dụng và các thông số tùy chỉnh, giúp người học và nhà phát triển dễ dàng tiếp cận và áp dụng vào thực tế. Ngoài ra, thư viện còn tương thích tốt với các thành phần khác trong hệ sinh thái Python khoa học như NumPy, SciPy và matplotlib, cho phép tích hợp mượt mà trong các quy trình phân tích dữ liệu và phát triển mô hình.

Trong quá trình học tập, chúng tôi khuyến khích bạn đọc đồng thời tham khảo tài liệu chính thức và hướng dẫn sử dụng tại trang chủ của scikit-learn (<https://scikit-learn.org>). Việc kết hợp giữa lý thuyết được trình bày trong sách này và tài nguyên trực tuyến sẽ giúp bạn xây dựng nền tảng vững chắc để hiểu và vận dụng hiệu quả các kỹ thuật học máy.

Trong các chương cuối của cuốn sách, chúng ta sẽ được tiếp tục khám phá cách sử dụng scikit-learn để xây dựng, huấn luyện và đánh giá các mô hình học máy thông qua các ví dụ minh họa cụ thể.

3.2.2. Các thư viện và công cụ hỗ trợ

Việc hiểu rõ vai trò và cách sử dụng **Scikit-learn** là một yếu tố cốt lõi trong quá trình học tập và thực hành học máy với Python. Tuy nhiên, để khai thác hiệu quả toàn bộ tiềm năng của Scikit-learn, người học cần nắm vững một số thư viện nền tảng và công cụ hỗ trợ khác trong hệ sinh thái Python.

Trước hết, cần lưu ý rằng Scikit-learn được xây dựng dựa trên hai thư viện khoa học cốt lõi là thư viện **NumPy** và **SciPy**:

- NumPy cung cấp các cấu trúc dữ liệu hiệu quả và toán tử đại số tuyến tính cần thiết cho việc thao tác trên mảng số liệu nhiều chiều.
- SciPy mở rộng chức năng của NumPy bằng cách cung cấp các thuật toán khoa học chuyên sâu như tối ưu hóa, tích phân số, nội suy và xử lý tín hiệu.

Bên cạnh đó, hai thư viện phổ biến khác sẽ thường xuyên được sử dụng trong quá trình học tập là **pandas** và **matplotlib**:

- pandas hỗ trợ cấu trúc dữ liệu dạng bảng (DataFrame) linh hoạt, giúp việc tổ chức, làm sạch và thao tác dữ liệu trở nên dễ dàng và trực quan hơn.

- matplotlib là một thư viện trực quan hóa dữ liệu mạnh mẽ, cho phép người học vẽ đồ thị, biểu đồ và hình ảnh trực quan hỗ trợ cho quá trình phân tích dữ liệu và đánh giá mô hình.

Cuối cùng, chúng ta sẽ sử dụng **Jupyter Notebook** – một môi trường lập trình tương tác dựa trên trình duyệt – cho phép kết hợp mã nguồn, kết quả thực thi, hình ảnh trực quan và ghi chú văn bản trong cùng một giao diện. Đây là công cụ lý tưởng để thực hành học máy, chia sẻ quy trình phân tích và trình bày kết quả.

Tóm lại, để tận dụng tối đa sức mạnh của Scikit-learn, người học nên nắm vững các công cụ nền tảng sau:

- NumPy
- SciPy
- pandas
- matplotlib
- Jupyter Notebook

Chúng sẽ là những thành phần quan trọng xuyên suốt toàn bộ nội dung của cuốn sách này, hỗ trợ bạn trong việc xây dựng, đánh giá và trực quan hóa các mô hình học máy một cách hiệu quả.

3.2.2.1. Thư viện NumPy

NumPy, viết tắt của *Numerical Python*, là một trong những thư viện cơ bản và thiết yếu nhất cho tính toán khoa học trong Python. Thư viện này cung cấp các cấu trúc dữ liệu và hàm toán học hiệu quả phục vụ cho xử lý mảng và ma trận nhiều chiều, đồng thời hỗ trợ các phép toán đại số tuyến tính, biến đổi Fourier, và sinh số ngẫu nhiên giả.

Trong ngữ cảnh của học máy với Scikit-learn, **mảng NumPy là cấu trúc dữ liệu cơ bản và bắt buộc**. Hầu hết các thuật toán trong Scikit-learn đều được thiết kế để làm việc với dữ liệu dưới dạng mảng NumPy. Do đó, trước khi huấn luyện hoặc đánh giá bất kỳ mô hình học máy nào, dữ liệu đầu vào phải được chuyển đổi về dạng mảng nhiều chiều của NumPy.

Cấu trúc trung tâm trong NumPy là lớp `ndarray`, viết tắt của n-dimensional array – một mảng có thể có từ một chiều trở lên, với tất cả các phần tử trong mảng phải thuộc cùng một kiểu dữ liệu. Lớp `ndarray` không chỉ hỗ trợ thao tác dữ liệu nhanh chóng và hiệu quả mà còn cung cấp khả năng tương tác tốt với các thư viện khác trong hệ sinh thái khoa học dữ liệu của Python.

Một mảng NumPy có thể có hình thức như sau:

```
import numpy as np

x = np.array([[1, 2, 3], [4, 5, 6]])
print("x:\n{}".format(x))
```

```
x:
[[1 2 3]
 [4 5 6]]
```

3.2.2.2. Thư viện SciPy

SciPy là một thư viện mở rộng được xây dựng trên nền tảng NumPy, cung cấp tập hợp các thuật toán và hàm toán học cao cấp nhằm phục vụ cho các nhu cầu tính toán khoa học chuyên sâu trong Python. SciPy bao gồm nhiều mô-đun chức năng, hỗ trợ các tác vụ như đại số tuyến tính nâng cao, tối ưu hóa hàm, xử lý tín hiệu, biến đổi Fourier, các hàm toán học đặc biệt và phân phối xác suất thống kê.

Trong bối cảnh học máy, Scikit-learn khai thác một số hàm và cấu trúc dữ liệu từ SciPy để triển khai các thuật toán học của mình. Do đó, SciPy có vai trò quan trọng trong việc hỗ trợ tính toán nền tảng và tối ưu hóa hiệu suất cho các mô hình học máy.

Một mô-đun đặc biệt đáng chú ý trong SciPy là `scipy.sparse`, cung cấp các **cấu trúc ma trận thưa**, hay *sparse matrices*. Ma trận thưa là một dạng biểu diễn dữ liệu hai chiều trong đó phần lớn các phần tử có giá trị bằng 0. Thay vì lưu trữ toàn bộ các giá trị bao gồm cả các số không cần thiết, `scipy.sparse` chỉ lưu trữ thông tin cần thiết về vị trí và giá trị của các phần tử khác không. Cách tiếp cận này giúp tiết kiệm đáng kể bộ nhớ và cải thiện hiệu năng khi làm việc với dữ liệu có mật độ thấp.

Trong Scikit-learn, ma trận thưa thường được sử dụng cho các bài toán như xử lý văn bản, nơi dữ liệu đầu vào là các vectơ đặc trưng có số chiều rất lớn nhưng phần lớn là các giá trị 0. Việc hiểu và sử dụng `scipy.sparse` là cần thiết để làm việc hiệu quả với những loại dữ liệu như vậy.

Ví dụ về tạo một ma trận thưa từ NumPy:

```
from scipy import sparse
import numpy as np

# Tạo một mảng NumPy 2D với đường chéo là các số một, và các số không ở mọi nơi khác
eye = np.eye(4)
print("Mảng NumPy: \n{}".format(eye))
```

```
Mảng NumPy:
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
```

```
# Chuyển đổi mảng NumPy thành ma trận thưa CSR của SciPy
# Chỉ các mục khác không được lưu trữ
sparse_matrix = sparse.csr_matrix(eye)
print("\nMa trận thưa CSR của SciPy:\n{}".format(sparse_matrix))
```

```
Ma trận thưa CSR của SciPy:
<Compressed Sparse Row sparse matrix of dtype 'float64'
with 4 stored elements and shape (4, 4)>
  Coords      Values
(0, 0)       1.0
(1, 1)       1.0
(2, 2)       1.0
(3, 3)       1.0
```

3.2.2.3. matplotlib

matplotlib là thư viện trực quan hóa dữ liệu khoa học tiêu chuẩn trong hệ sinh thái Python. Thư viện này cung cấp các công cụ linh hoạt và mạnh mẽ để tạo ra các hình ảnh chất lượng cao, phù hợp cho cả mục đích phân tích dữ liệu lẫn xuất bản học thuật. Với matplotlib, người dùng có thể dễ dàng xây dựng các biểu đồ đường, biểu đồ cột, biểu đồ phân tán, biểu đồ hộp, biểu đồ mật độ, và nhiều loại hình ảnh trực quan khác.

Trong quy trình phân tích dữ liệu, việc trực quan hóa đóng vai trò không thể thiếu vì nó giúp khám phá cấu trúc dữ liệu, phát hiện xu hướng, mô hình và bất thường tiềm ẩn mà các thống kê mô tả đơn thuần có thể không thể hiện được. Trong cuốn sách này, chúng ta sẽ sử dụng

matplotlib như là công cụ chính để hiển thị và minh họa dữ liệu trong suốt quá trình học tập và thực hành.

Khi làm việc trong môi trường Jupyter Notebook, bạn có thể tích hợp trực tiếp hình ảnh vào trình duyệt bằng cách sử dụng các lệnh ma thuật (magic commands) của IPython. Cụ thể:

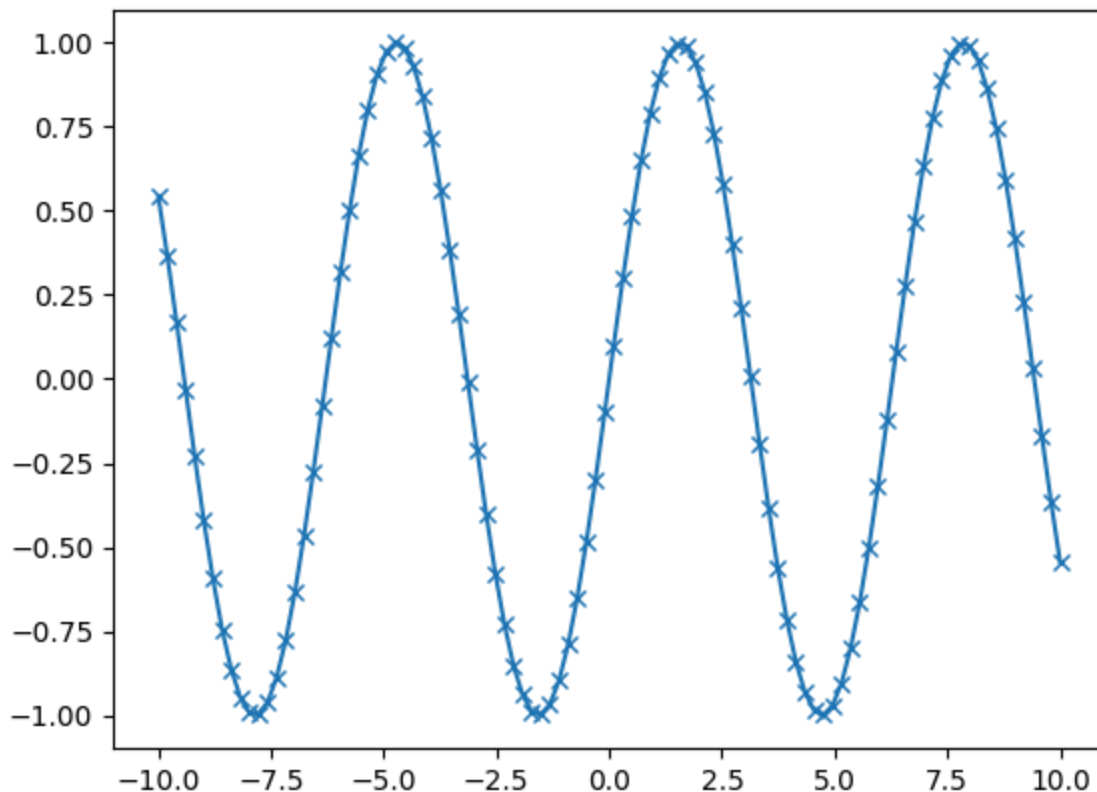
- `%matplotlib inline` hiển thị biểu đồ dưới dạng ảnh tĩnh ngay bên dưới ô lệnh.
- `%matplotlib notebook` cho phép tạo biểu đồ tương tác, hỗ trợ các thao tác như phóng to, xoay, hoặc di chuyển trực tiếp trong trình duyệt.

Chúng tôi khuyến nghị sử dụng `%matplotlib notebook` khi có thể, nhằm tận dụng tính năng tương tác nâng cao trong việc khám phá và trình bày dữ liệu. Dưới đây là một ví dụ về tạo đồ thị bằng `matplotlib`

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

# Tạo một chuỗi số từ -10 đến 10 với 100 bước ở giữa
x = np.linspace(-10, 10, 100)
# Tạo một mảng thứ hai sử dụng sin
y = np.sin(x)
# Hàm plot tạo một biểu đồ đường của một mảng so với một mảng khác
plt.plot(x, y, marker="x")
```

```
[<matplotlib.lines.Line2D at 0x2a1864ca710>]
```



3.2.2.4. Thư viện pandas

pandas là một thư viện mạnh mẽ trong Python được thiết kế để hỗ trợ thao tác và phân tích dữ liệu có cấu trúc. Trọng tâm của pandas là cấu trúc dữ liệu **DataFrame**, được mô hình hóa theo khái niệm DataFrame trong ngôn ngữ R. Về cơ bản, một DataFrame là một bảng dữ liệu hai chiều, tương tự như bảng tính trong Excel hoặc một bảng trong cơ sở dữ liệu quan hệ.

pandas cung cấp một tập hợp phong phú các phương thức hỗ trợ thao tác với dữ liệu, bao gồm lọc, gộp nhóm, chuyển định dạng, xử lý thiếu dữ liệu, và nhiều chức năng tiền xử lý phổ biến khác. Đặc biệt, DataFrame trong pandas hỗ trợ các truy vấn dữ liệu có cú pháp gần giống với SQL, cũng như các phép kết hợp bảng linh hoạt và hiệu quả.

Khác với NumPy – nơi các mảng ndarray yêu cầu tất cả phần tử phải có cùng kiểu dữ liệu – pandas cho phép mỗi cột trong `DataFrame` có kiểu dữ liệu riêng biệt (chẳng hạn: số nguyên, số thực, chuỗi, kiểu ngày giờ). Điều này đặc biệt hữu ích khi làm việc với dữ liệu thực tế, nơi các thuộc tính thường có kiểu dữ liệu không đồng nhất.

Một trong những tính năng đáng giá khác của pandas là khả năng **nhập dữ liệu từ nhiều nguồn phổ biến**, bao gồm:

- Tập CSV
- Tập Excel
- Cơ sở dữ liệu quan hệ
- Tập JSON, Parquet, v.v.

pandas đóng vai trò quan trọng trong hầu hết các quy trình xử lý dữ liệu và là thành phần không thể thiếu trong chuỗi công cụ học máy với Python.

Ví dụ đơn giản sau đây minh họa cách tạo một DataFrame từ một đối tượng kiểu từ điển trong Python:

```
import pandas as pd
from IPython.display import display

# tạo một tập dữ liệu đơn giản về mọi người
data = {'Name': ["John", "Anna", "Peter", "Linda"],
        'Location': ["New York", "Paris", "Berlin", "London"],
        'Age': [24, 13, 53, 33]}

data_pandas = pd.DataFrame(data)
# IPython.display cho phép "in đẹp" các dataframe
# trong Jupyter notebook
display(data_pandas)
```

	Name	Location	Age
0	John	New York	24
1	Anna	Paris	13
2	Peter	Berlin	53
3	Linda	London	33

3.3. Ứng dụng thực tế của mô hình học máy

Việc tiếp cận các mô hình học máy thông qua ví dụ thực tế không chỉ giúp người học hiểu rõ bản chất của thuật toán mà còn rút ngắn khoảng cách giữa lý thuyết và ứng dụng. Thay vì bắt đầu bằng các công thức trừu tượng hoặc mô hình tổng quát, việc quan sát cách một thuật toán xử lý dữ liệu thực — như dự đoán giá nhà hay phân loại khách hàng — sẽ tạo điều kiện cho

người học hình thành trực giác và nhận diện được ý nghĩa thực tiễn của các bước trong quy trình học máy. Trong phần tiếp theo, chúng tôi sẽ giới thiệu nhiều ví dụ cụ thể để minh họa cách xây dựng, huấn luyện và đánh giá mô hình, từ đó củng cố kiến thức lý thuyết thông qua trải nghiệm trực tiếp.

3.3.1. Phân loại các loài hoa Iris

Trong phần này, chúng ta sẽ cùng tìm hiểu một ứng dụng học máy đơn giản và xây dựng mô hình đầu tiên. Thông qua ví dụ minh họa, một số khái niệm và thuật ngữ cốt lõi trong học máy sẽ được giới thiệu một cách trực quan và dễ tiếp cận.

Giả sử bạn là một nhà thực vật học nghiệp dư quan tâm đến việc phân biệt các loài hoa *Iris* bắt gặp trong tự nhiên. Nhằm phục vụ mục tiêu phân loại, bạn đã tiến hành thu thập một số đặc trưng định lượng của từng bông hoa, bao gồm: chiều dài và chiều rộng của cánh hoa (*petal length* và *petal width*), cùng với chiều dài và chiều rộng của đài hoa (*sepal length* và *sepal width*). Tất cả các giá trị đều được đo bằng đơn vị centimet.

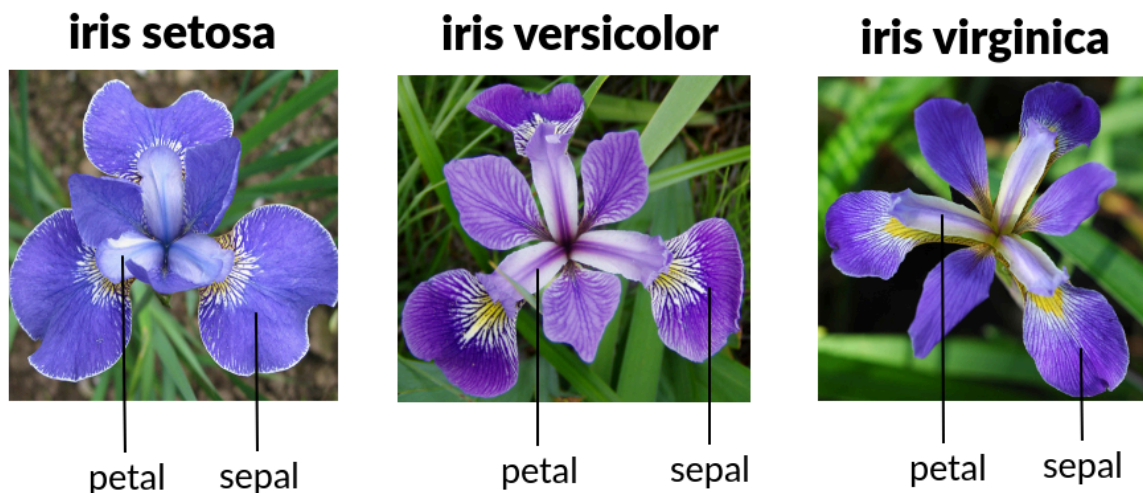


Fig. 3.2 Phân loại hoa iris dựa trên các thuộc tính là độ dài và độ rộng của các cánh hoa

Bên cạnh các quan sát thực tế, bạn còn có trong tay một tập dữ liệu tham chiếu bao gồm các đặc trưng tương tự của những bông hoa *Iris* đã được phân loại chính xác bởi các nhà thực vật học chuyên nghiệp. Tập dữ liệu này bao gồm ba loài hoa phổ biến: **Iris setosa**, **Iris versicolor**, và **Iris virginica**. Giả định rằng đây là ba loài duy nhất sẽ xuất hiện trong quá trình quan sát ngoài thực địa.

Mục tiêu đặt ra là xây dựng một mô hình học máy có khả năng học từ tập dữ liệu đã được gán nhãn – nghĩa là với mỗi mẫu dữ liệu, loài của bông hoa đã được xác định rõ – từ đó dự đoán chính xác loài của một bông hoa mới chỉ dựa trên các đặc trưng hình thái đã được đo đạc.

Ví dụ này là một trường hợp điển hình của **bài toán phân loại có giám sát** (*supervised classification*), trong đó:

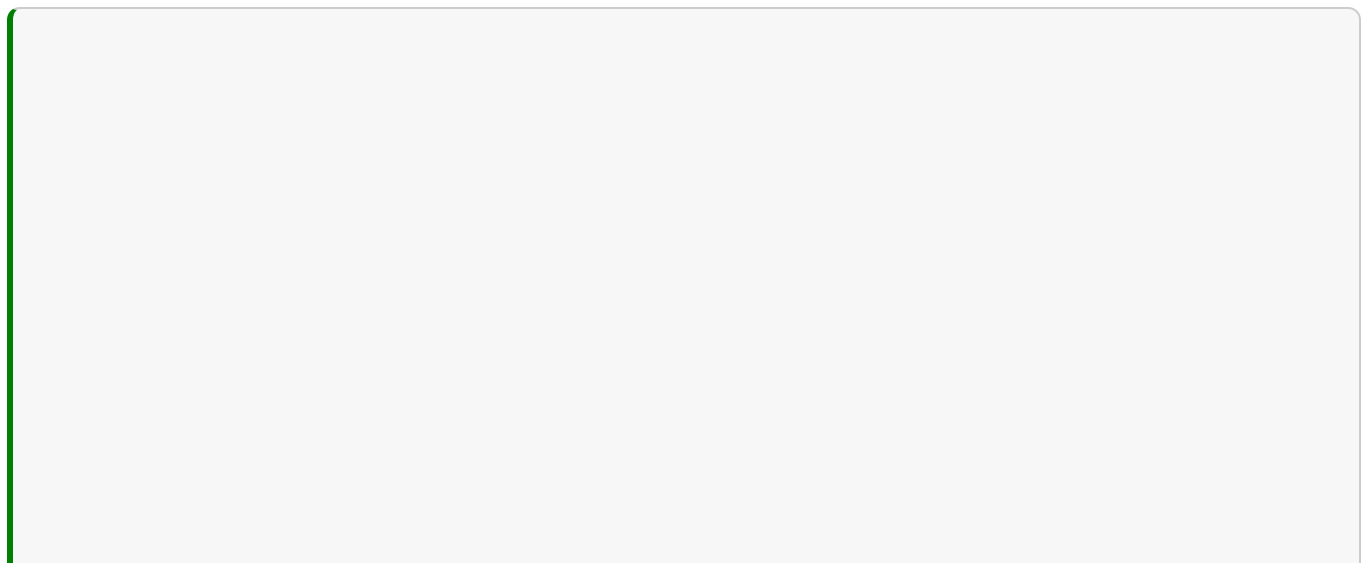
- **Đầu vào** là một tập hợp các đặc trưng định lượng, trong dữ liệu iris là các phép đo về hình thái của hoa.
- **Đầu ra** là một **nhãn** rời rạc, là loài của hoa iris trong dữ liệu.

Vì dữ liệu huấn luyện đã cung cấp thông tin về đầu ra mong muốn, đây là một bài toán **học có giám sát**. Trong bối cảnh này, nhãn là loài của bông hoa, còn các giá trị nhãn khả dĩ (Iris setosa, Iris versicolor, Iris virginica) được gọi là các **lớp**, hay các *classes*. Mỗi mẫu hoa trong tập dữ liệu huấn luyện thuộc về đúng một trong ba lớp này, do đó bài toán này là một ví dụ về bài toán phân loại nhiều lớp.

Trong các phần tiếp theo, chúng ta sẽ sử dụng tập dữ liệu *Iris* để huấn luyện mô hình phân loại, đánh giá độ chính xác và thảo luận các kỹ thuật đánh giá hiệu suất mô hình cơ bản.

3.3.1.1. Làm quen với dữ liệu phân loại hoa iris

Dữ liệu chúng ta sẽ sử dụng cho ứng dụng này là tập dữ liệu Iris, một tập dữ liệu đơn giản và kinh điển trong xây dựng mô hình học máy và thống kê. Dữ liệu này đã được chứa sẵn trong thư viện `scikit-learn` trong mô-đun `datasets`. Chúng ta có thể tải dữ liệu bằng cách gọi hàm `load_iris`:



```

from sklearn.datasets import load_iris
import pandas as pd

# Tải dữ liệu iris
iris_dataset = load_iris()

# Tạo DataFrame từ dữ liệu và tên cột
df = pd.DataFrame(data=iris_dataset.data, columns=iris_dataset.feature_names)

# Thêm cột target (nhân loại)
df["target"] = iris_dataset.target

# (Tùy chọn) Thêm tên loài (setosa, versicolor, virginica)
df["species"] = pd.Categorical.from_codes(iris_dataset.target, iris_dataset.target_names)

# Xem dữ liệu
df

```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	species
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa
...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2	virginica
146	6.3	2.5	5.0	1.9	2	virginica
147	6.5	3.0	5.2	2.0	2	virginica
148	6.2	3.4	5.4	2.3	2	virginica
149	5.9	3.0	5.1	1.8	2	virginica

150 rows × 6 columns

Dữ liệu có tên `df` chứa các phép đo định lượng tương ứng với **150 bông hoa** khác nhau, trong đó mỗi hàng đại diện cho một **mẫu** (*sample*), và mỗi cột đại diện cho một **đặc trưng** (*feature*) mô tả thuộc tính hình thái của bông hoa. Cụ thể, mỗi mẫu bao gồm bốn đặc trưng: chiều dài và

chiều rộng của cánh hoa (*petal length*, *petal width*), và chiều dài và chiều rộng của đài hoa (*sepal length*, *sepal width*), được đo bằng đơn vị centimet.

Cấu trúc của mảng dữ liệu tuân theo quy ước chung trong scikit-learn: một mảng hai chiều với hình dạng `(n_samples, n_features)`, tức là số lượng mẫu nhân với số lượng đặc trưng. Trong trường hợp của tập dữ liệu *Iris*, mảng dữ liệu có kích thước `(150, 4)`, nghĩa là có 150 mẫu và mỗi mẫu có 4 đặc trưng.

Ví dụ, khi quan sát năm mẫu đầu tiên, ta thấy rằng tất cả đều có chiều rộng cánh hoa là 0.2 cm, và trong đó bông hoa đầu tiên có đài hoa dài nhất với chiều dài 5.1 cm. Những quan sát sơ bộ này cho phép người học bắt đầu hình dung mối quan hệ giữa các đặc trưng hình thái và loài hoa.

Song song với 4 đặc trưng, dữ liệu cũng cung cấp cột `target`, là một mảng một chiều có chiều dài bằng số mẫu, trong đó mỗi phần tử là một nhãn số nguyên đại diện cho loài của bông hoa tương ứng trong cột `species`. Cụ thể:

- Nhãn 0 tương ứng với *setosa*
- Nhãn 1 tương ứng với *versicolor*
- Nhãn 2 tương ứng với *virginica*

Như vậy, mảng `target` đóng vai trò là nhãn giám sát, hay *labels*, trong bài toán phân loại có giám sát. Việc hiểu rõ cấu trúc này là điều kiện tiên quyết để xử lý và huấn luyện các mô hình học máy trên dữ liệu *Iris* một cách chính xác.

3.3.1.2. Dữ liệu huấn luyện mô hình và dữ liệu kiểm tra mô hình

Mục tiêu của chúng ta là xây dựng một mô hình học máy có khả năng dự đoán loài *Iris* dựa trên các phép đo hình thái học. Tuy nhiên, trước khi áp dụng mô hình cho các dữ liệu mới, một câu hỏi quan trọng cần được đặt ra: *Làm thế nào để biết mô hình có hoạt động hiệu quả không?*

Một sai lầm phổ biến là **đánh giá hiệu suất của mô hình bằng chính dữ liệu đã được sử dụng để huấn luyện nó**. Cách tiếp cận này dễ dẫn đến hiện tượng **khớp quá mức**, hay còn gọi là *overfitting*, trong đó mô hình chỉ đơn giản là ghi nhớ toàn bộ tập huấn luyện thay vì học cách **tổng quát hóa từ dữ liệu**. Kết quả là, mô hình có thể đạt độ chính xác tuyệt đối trên dữ liệu huấn luyện nhưng lại hoạt động kém khi gặp dữ liệu mới.

Để đánh giá khách quan hiệu suất của mô hình, ta cần kiểm tra nó trên những **dữ liệu mà nó chưa từng thấy trước đó** — nhưng chúng ta vẫn cần có nhãn để so sánh kết quả dự đoán. Thực tế này dẫn đến một quy trình chuẩn trong học máy: chia tập dữ liệu có nhãn thành hai phần:

- **Tập huấn luyện** (*training set*): Dùng để xây dựng mô hình.
- **Tập kiểm tra** (*test set*): Dùng để đánh giá mô hình sau khi huấn luyện.

Trong scikit-learn, quá trình này được hỗ trợ bởi hàm `train_test_split`, cho phép người dùng dễ dàng xáo trộn và phân chia dữ liệu một cách ngẫu nhiên. Theo thông lệ phổ biến, khoảng 50% đến 80% dữ liệu được sử dụng để huấn luyện và 50% đến 20% còn lại dành cho kiểm tra. Nguyên tắc phân chia được điều chỉnh tùy theo mục tiêu và kích thước của tập dữ liệu.

Một lưu ý quan trọng trước khi chia dữ liệu là chúng ta cần phải **trộn toàn bộ dữ liệu một cách ngẫu nhiên** để đảm bảo sự đa dạng và đại diện của cả ba lớp, bao gồm *setosa*, *versicolor*, *virginica*, trong cả hai tập huấn luyện và kiểm tra. Điều này là cần thiết bởi vì dữ liệu *Iris* mặc định được sắp xếp theo nhãn. Nếu chỉ lấy 30% cuối của dữ liệu làm tập kiểm tra, toàn bộ mẫu kiểm tra sẽ chỉ chứa một lớp duy nhất, chẳng hạn toàn bộ là *Iris virginica*, dẫn đến việc đánh giá sai lệch và không phản ánh đúng khả năng tổng quát hóa của mô hình.

Hàm `train_test_split` thực hiện việc trộn dữ liệu một cách ngẫu nhiên. Để đảm bảo khả năng kiểm soát kết quả thực nghiệm, ta có thể truyền một giá trị cụ thể vào tham số `random_state` trong hàm `train_test_split` để cố định bộ tạo số ngẫu nhiên với mục đích tái lập việc chia dữ liệu trong các lần chạy sau:

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    df[["sepal length (cm)", "sepal width (cm)", "petal length (cm)", "petal width (cm)"],
    random_state = 10, test_size=0.3)

Y_train = pd.Categorical(Y_train)
Y_test = pd.Categorical(Y_test)
```

```
print("Hình dạng X_train: {}".format(X_train.shape))
print("Hình dạng Y_train: {}".format(Y_train.shape))
```

```
Hình dạng X_train: (105, 4)
Hình dạng Y_train: (105,)
```

```
print("Hình dạng X_test: {}".format(X_test.shape))
print("Hình dạng Y_test: {}".format(Y_test.shape))
```

```
Hình dạng X_test: (45, 4)
Hình dạng Y_test: (45,)
```

3.3.1.3. Phân tích dữ liệu khám phá (EDA)

Trước khi tiến hành xây dựng một mô hình học máy, việc kiểm tra và khám phá dữ liệu là một bước quan trọng không thể bỏ qua. Quá trình này không chỉ giúp xác định mức độ phù hợp của dữ liệu đối với bài toán học máy mà còn hỗ trợ đánh giá xem liệu bài toán có thể được giải quyết bằng các phương pháp phân tích đơn giản hơn, mà không cần đến mô hình hóa phức tạp.

Một mục tiêu khác của bước khám phá dữ liệu là phát hiện ra các điểm bất thường hoặc sai lệch tiềm ẩn. Trong các ứng dụng thực tế, dữ liệu thu thập thường không hoàn hảo: có thể chứa giá trị thiếu, lỗi đo lường, hoặc sự không nhất quán về đơn vị. Ví dụ, một số mẫu hoa *Iris* có thể đã được đo bằng inch thay vì centimet, gây ảnh hưởng nghiêm trọng đến kết quả phân tích nếu không được phát hiện và xử lý kịp thời.

Một trong những phương pháp hiệu quả nhất để khám phá dữ liệu là thông qua trực quan hóa. Cách tiếp cận đơn giản và phổ biến nhất là sử dụng **biểu đồ phân tán**, trong đó một đặc trưng được đặt trên trục hoành và một đặc trưng khác trên trục tung. Mỗi điểm dữ liệu sẽ được biểu diễn bằng một dấu chấm trên mặt phẳng hai chiều, cho phép người quan sát nhận diện trực tiếp các mối quan hệ tuyến tính, cụm dữ liệu, hoặc ngoại lệ.

Tuy nhiên, do hạn chế của không gian hiển thị hai chiều, việc trực quan hóa toàn bộ tập dữ liệu nhiều chiều là điều không khả thi. Một giải pháp khả thi trong trường hợp số lượng đặc trưng còn tương đối nhỏ là sử dụng **biểu đồ cặp**, hay còn gọi là *pair plot*. Biểu đồ này hiển thị tất cả các cặp kết hợp có thể giữa các đặc trưng, cho phép người dùng quan sát các mối liên hệ hai chiều giữa từng cặp biến. Trong trường hợp tập dữ liệu Iris, với chỉ bốn đặc trưng, phương pháp này hoàn toàn phù hợp.

Tuy nhiên, cần lưu ý rằng biểu đồ cặp chỉ thể hiện mối quan hệ giữa hai đặc trưng tại một thời điểm. Do đó, những tương tác phức tạp hơn liên quan đến ba hoặc nhiều đặc trưng cùng lúc có thể không được thể hiện đầy đủ qua hình thức trực quan này. Điều này nhấn mạnh tầm quan trọng của việc kết hợp nhiều phương pháp khám phá dữ liệu khác nhau, bao gồm cả phân tích thống kê và kỹ thuật giảm chiều, nhằm hiểu rõ hơn về cấu trúc và hành vi của tập dữ liệu trước khi áp dụng các thuật toán học máy.

Trước tiên chúng ta xem một vài thống kê mô tả của các đặc trưng và biến mục tiêu trong dữ liệu huấn luyện:

```
X_train.groupby(Y_train).agg(["mean", "std", "min", "max"])
```

	sepal length (cm)				sepal width (cm)				petal len
	mean	std	min	max	mean	std	min	max	mean
setosa	5.047222	0.374537	4.3	5.8	3.480556	0.371729	2.9	4.4	1.463889
versicolor	5.948485	0.539746	4.9	7.0	2.766667	0.293329	2.2	3.2	4.209091
virginica	6.541667	0.650879	4.9	7.9	2.983333	0.322047	2.2	3.8	5.538889

```
Y_train.describe()
```

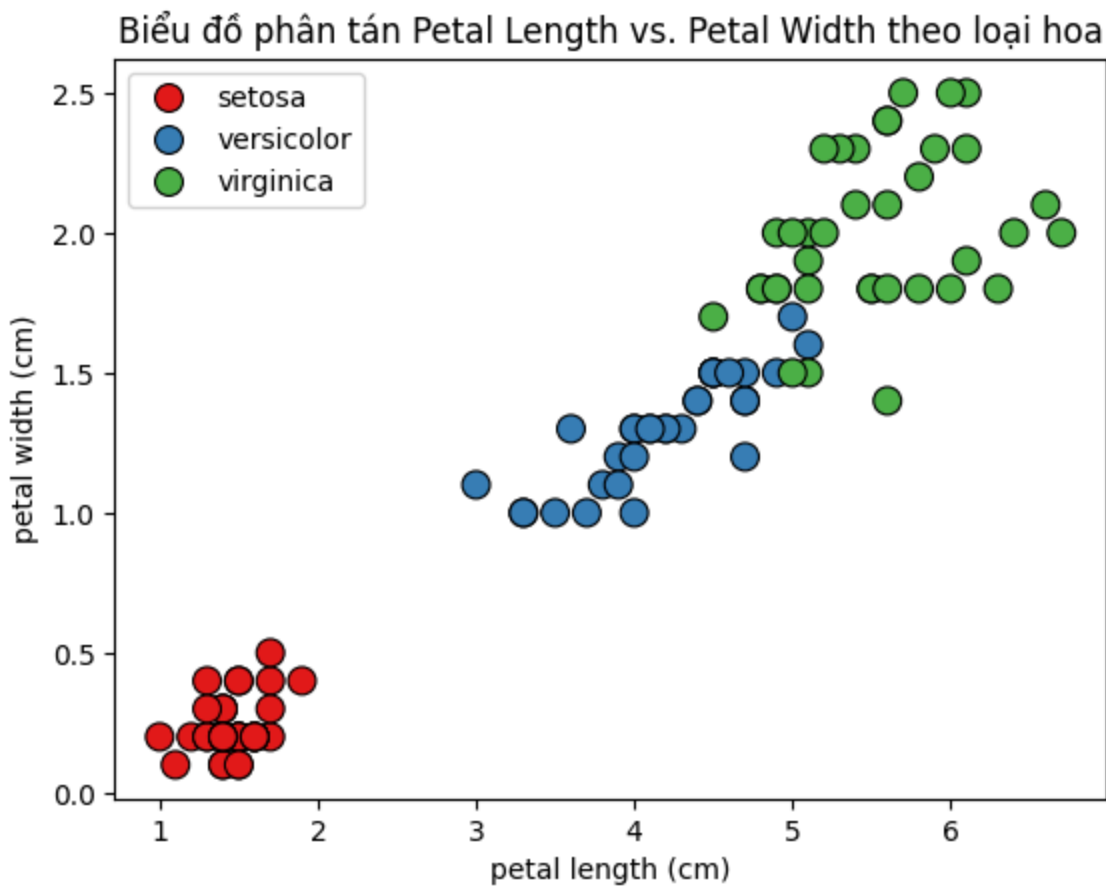
	counts	freqs
categories		
setosa	36	0.342857
versicolor	33	0.314286
virginica	36	0.342857

Có thể thấy rằng có sự khác biệt tương đối giữa các đặc trưng của ba loại hoa trong dữ liệu. Hai đặc trưng có sự phân biệt rõ nhất là **petal length** và **petal width**. Tuy nhiên thống kê mô tả chỉ cho chúng ta cảm nhận về từng biến riêng lẻ mà không nhìn nhận được mối liên hệ giữa các đặc trưng. Một cách trực quan để đánh giá mối liên hệ giữa các đặc trưng của từng loại hoa iris là trực quan hóa sử dụng đồ thị phân tán:


```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    data= X_train,
    x="petal length (cm)",
    y="petal width (cm)",
    hue = Y_train,
    palette = "Set1",
    s = 100, # size điểm
    edgecolor='k'
)

plt.title("Biểu đồ phân tán Petal Length vs. Petal Width theo loại hoa")
plt.show()
```



Quan sát từ các biểu đồ trực quan cho thấy rằng ba lớp loài hoa có xu hướng được phân tách khá rõ ràng thông qua các đặc trưng hình thái của đài hoa. Điều này gợi ý rằng một mô hình học máy nhiều khả năng sẽ có thể học được các ranh giới phân biệt hiệu quả giữa các lớp dựa trên các đặc trưng này. Bạn đọc có thể tiếp tục trực quan hóa các đặc trưng khác để tìm ra các đặc trưng quan trọng khác.

Nhìn chung, quá trình phân tích dữ liệu khám phá đóng vai trò đặc biệt quan trọng trong việc lựa chọn các đặc trưng phù hợp để đưa vào mô hình học máy. Việc hiểu rõ cấu trúc, mối quan hệ và phân phối của các biến trong dữ liệu không chỉ giúp phát hiện các bất thường tiềm ẩn mà còn là cơ sở để xây dựng biểu diễn đầu vào hiệu quả. Chính chất lượng của bước lựa chọn đặc trưng này có ảnh hưởng trực tiếp đến hiệu suất tổng thể và khả năng khái quát hóa của mô hình.

Do đây chỉ là một ví dụ đơn giản, với ít đặc trưng, nên chúng tôi không đi sâu vào quá trình này mà sẽ sử dụng cả bốn đặc trưng ban đầu của dữ liệu để xây dựng mô hình

3.3.1.4. Mô hình đầu tiên: k-láng giềng gần nhất

Sau khi hoàn tất quá trình khám phá và tiền xử lý dữ liệu, chúng ta có thể bắt đầu xây dựng mô hình học máy đầu tiên. Thư viện `scikit-learn` cung cấp nhiều thuật toán phân loại khác nhau, và trong phần này, chúng ta sẽ sử dụng một trong những thuật toán cơ bản và dễ hiểu nhất: phân loại bằng mô hình k-láng giềng gần nhất, hay *k-nearest neighbors classification*.

Mô hình k-láng giềng gần nhất là mô hình **phi tham số** vì không có một dạng tham số cụ thể cho mối liên hệ giữa các đặc trưng và biến mục tiêu. Ý tưởng cốt lõi của thuật toán này là rất trực quan: quá trình huấn luyện không liên quan đến việc tìm kiếm các tham số, mà chỉ đơn giản là lưu trữ toàn bộ tập huấn luyện. Khi cần dự đoán nhãn cho một điểm dữ liệu mới, thuật toán sẽ tìm các điểm trong tập huấn luyện có khoảng cách gần nhất với điểm mới đó, sau đó gán nhãn của đa số các điểm gần nhất làm kết quả dự đoán.

Tham số `k` trong tên gọi thuật toán đề cập đến số lượng điểm láng giềng được xem xét. Thay vì chỉ sử dụng một hàng xóm gần nhất, ta có thể xét `k` hàng xóm gần nhất, ví dụ là 3 hoặc 5, hoặc nhiều hơn. Nhãn được dự đoán sẽ là nhãn chiếm đa số trong số các hàng xóm này. Việc tăng `k` giúp làm giảm độ nhạy của mô hình với nhiễu hoặc các điểm bất thường trong dữ liệu huấn luyện. Trong ví dụ hiện tại, để đơn giản hóa, chúng ta sẽ sử dụng `k = 3`, tức là chỉ xét ba láng giềng gần nhất.

Tất cả các thuật toán học máy trong `scikit-learn` đều được tổ chức dưới dạng các lớp riêng biệt, được gọi là **Estimator**, đại diện cho các mô hình học có thể huấn luyện và dự đoán. Thuật toán k- điểm láng giềng gần nhất được triển khai trong lớp `KNeighborsClassifier`, thuộc mô-đun `sklearn.neighbors`.

Trước khi sử dụng mô hình, chúng ta cần khởi tạo một instance của lớp `KNeighborsClassifier`. Tại thời điểm khởi tạo, các siêu tham số của mô hình có thể được thiết lập — trong trường hợp này là số lượng hàng xóm (`n_neighbors`), mà chúng ta sẽ đặt bằng $k = 3$:

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3)
```

Sau khi khởi tạo đối tượng `knn` từ lớp `KNeighborsClassifier`, đối tượng này sẽ đóng vai trò là mô hình học máy, bao gồm cả thuật toán được sử dụng để huấn luyện mô hình từ dữ liệu và thuật toán để dự đoán trên các điểm dữ liệu mới. Bên cạnh đó, đối tượng còn lưu giữ các thông tin được trích xuất từ tập huấn luyện trong quá trình huấn luyện. Trong trường hợp của `KNeighborsClassifier`, thông tin được lưu trữ đơn giản là toàn bộ tập dữ liệu huấn luyện, do thuật toán không thực hiện quá trình học tham số rõ ràng mà chỉ dựa trên khoảng cách đến các điểm đã biết.

Để thực hiện quá trình huấn luyện, ta sử dụng phương thức `.fit()` của đối tượng mô hình. Phương thức này nhận vào hai đối số:

- `X_train`: một mảng NumPy hai chiều chứa dữ liệu huấn luyện, trong đó mỗi hàng tương ứng với một mẫu và mỗi cột là một đặc trưng.
- `Y_train`: một mảng một chiều chứa nhãn tương ứng cho mỗi mẫu trong `X_train`.

Lệnh huấn luyện được thực hiện như sau:

```
knn.fit(X_train, Y_train)
```

▼ `KNeighborsClassifier` ⓘ ?
`KNeighborsClassifier(n_neighbors=3)`

Phương thức `.fit()` trong scikit-learn không chỉ thực hiện việc huấn luyện mô hình mà còn trả về chính đối tượng mô hình đã được huấn luyện, đồng thời sửa đổi nó tại chỗ. Do đó, khi gọi `fit`, ta nhận được một biểu diễn chuỗi của đối tượng mô hình, cho biết cấu hình và các tham số đã được sử dụng trong quá trình khởi tạo.

Ví dụ, sau khi gọi `knn.fit(X_train, Y_train)`, nếu in ra đối tượng `knn`, bạn sẽ thấy một chuỗi hiển thị tên lớp (`KNeighborsClassifier`) cùng với các tham số tương ứng. Trong số đó, tham số `n_neighbors=3` sẽ được liệt kê rõ ràng, vì đây là giá trị mà người dùng đã chỉ định. Các tham số

còn lại thường giữ nguyên giá trị mặc định, được thiết kế để hoạt động tốt trong đa số trường hợp hoặc dành cho các tình huống sử dụng đặc biệt.

Mặc dù biểu diễn chuỗi của một mô hình scikit-learn có thể trông khá dài và phức tạp, bạn đọc không cần cảm thấy bị choáng ngợp. Phần lớn các tham số mặc định là các tối ưu hóa liên quan đến tốc độ hoặc độ ổn định, và chỉ một số ít trong số đó có ảnh hưởng đáng kể đến hành vi của mô hình. Các tham số quan trọng và thường được điều chỉnh sẽ được trình bày chi tiết trong các chương liên quan đến xây dựng mô hình.

Sau khi mô hình đã được huấn luyện, chúng ta có thể sử dụng nó để dự đoán loài hoa cho các điểm dữ liệu mới mà chưa được gán nhãn. Đây chính là mục tiêu cuối cùng của một mô hình phân loại: khả năng khái quát hóa để đưa ra dự đoán chính xác cho dữ liệu chưa từng gặp trước đó.

Giả sử trong một nghiên cứu thực địa, chúng ta bắt gặp một bông hoa *Iris* mới và tiến hành đo các đặc trưng hình thái học như sau:

- Chiều dài đài hoa (*sepal length*): 5.0 cm
- Chiều rộng đài hoa (*sepal width*): 2.9 cm
- Chiều dài cánh hoa (*petal length*): 1.0 cm
- Chiều rộng cánh hoa (*petal width*): 0.2 cm

Mục tiêu là xác định *loài của bông hoa này dựa trên các đặc trưng nêu trên.

Trước khi có thể đưa điểm dữ liệu mới này vào mô hình để dự đoán, chúng ta cần đảm bảo rằng nó có đúng định dạng đầu vào như dữ liệu huấn luyện: một mảng NumPy hai chiều với hình dạng `(1, 4)`, tương ứng với một mẫu và bốn đặc trưng.

Dưới đây là cách biểu diễn mẫu mới dưới dạng mảng:

```
X_new = np.array([[5, 2.9, 1, 0.2]])
```

Để đưa ra dự đoán, chúng ta gọi phương thức `predict` của đối tượng `knn`:

```
prediction = knn.predict(X_new)
print("Dự đoán: {}".format(prediction))
```

Dự đoán: ['setosa']

Khi áp dụng mô hình học máy đã huấn luyện để phân loại một điểm dữ liệu mới, chẳng hạn như bông hoa *Iris* có các đặc trưng được nêu ở trên, mô hình dự đoán rằng mẫu này thuộc lớp 0, tương ứng với loài *Iris setosa*. Tuy nhiên, việc mô hình đưa ra một nhãn dự đoán không đồng nghĩa với việc chúng ta có thể hoàn toàn tin tưởng vào kết quả này.

Một câu hỏi thiết yếu trong học máy là: *Làm thế nào để biết liệu mô hình có đưa ra dự đoán đáng tin cậy hay không?*

Trong thực tế, với các điểm dữ liệu mới được thu thập, chúng ta không biết nhãn thực tế – chính vì lý do này mà ta mới cần đến mô hình học máy để suy luận. Do đó, việc đánh giá hiệu suất mô hình không thể dựa trên các dự đoán đơn lẻ cho dữ liệu không có nhãn.

Thay vào đó, để kiểm tra và đánh giá độ chính xác tổng thể của mô hình, ta sử dụng một phần dữ liệu đã được gán nhãn nhưng **không tham gia vào quá trình huấn luyện**. Đây chính là vai trò của **tập kiểm tra** trong quy trình học có giám sát. Việc đánh giá mô hình trên tập kiểm tra sẽ giúp ta đo lường được khả năng tổng quát hóa của mô hình — tức là, mô hình có thể áp dụng tốt đến mức nào với dữ liệu mới mà nó chưa từng thấy trong quá trình huấn luyện.

Đây là lúc tập kiểm tra mà chúng ta đã tạo trước đó phát huy tác dụng. Dữ liệu này không được sử dụng để xây dựng mô hình, nhưng chúng ta biết loài chính xác của mỗi bông hoa iris trong tập kiểm tra.

Do đó, chúng ta có thể đưa ra dự đoán cho mỗi bông hoa iris trong dữ liệu kiểm tra và so sánh nó với nhãn của nó. Chúng ta có thể đo lường mô hình hoạt động tốt như thế nào bằng cách tính toán độ chính xác, là tỷ lệ phần trăm các bông hoa được dự đoán đúng loài:

```
Y_pred = knn.predict(X_test)
print("Dự đoán tập kiểm tra:\n {}".format(Y_pred))
```

Dự đoán tập kiểm tra:

```
['versicolor' 'virginica' 'setosa' 'versicolor' 'setosa' 'versicolor'
'virginica' 'versicolor' 'setosa' 'versicolor' 'versicolor' 'virginica'
'versicolor' 'setosa' 'setosa' 'virginica' 'versicolor' 'setosa' 'setosa'
'setosa' 'virginica' 'virginica' 'virginica' 'setosa' 'versicolor'
'setosa' 'versicolor' 'versicolor' 'versicolor' 'virginica' 'versicolor'
'versicolor' 'virginica' 'virginica' 'virginica' 'setosa' 'virginica'
'virginica' 'virginica' 'virginica' 'setosa' 'setosa' 'versicolor'
'setosa' 'versicolor']
```

```
print("Điểm tập kiểm tra: {:.2f}".format(np.mean(Y_pred == Y_test)))
```

Điểm tập kiểm tra: 0.98

Chúng ta cũng có thể sử dụng phương thức `score` của đối tượng `knn`. Phương thức này sẽ tính toán độ chính xác của tập kiểm tra cho chúng ta:

```
print("Điểm tập kiểm tra: {:.2f}".format(knn.score(X_test, Y_test)))
```

Điểm tập kiểm tra: 0.98

Sau khi áp dụng mô hình đã huấn luyện lên tập dữ liệu kiểm tra, chúng ta thu được độ chính xác, hay *accuracy*, xấp xỉ **0.98**, nghĩa là mô hình đã dự đoán chính xác nhãn cho khoảng **98% số mẫu** trong tập kiểm tra. Chỉ số này phản ánh mức độ tương đồng giữa nhãn dự đoán và nhãn thực tế của các mẫu mà mô hình chưa từng được tiếp xúc trong quá trình huấn luyện.

Dưới một số giả định nhất định về tính đại diện của tập kiểm tra và phân phối dữ liệu, độ chính xác này có thể được hiểu như một ước lượng cho khả năng mô hình sẽ hoạt động chính xác trên các điểm dữ liệu mới. Cụ thể, chúng ta có thể kỳ vọng rằng mô hình sẽ đưa ra dự đoán đúng khoảng 98% số lần khi được áp dụng vào các bông hoa *Iris* mới trong điều kiện tương tự.

Trong bối cảnh ứng dụng thực tế của một nhà thực vật học nghiệp dư, mức độ chính xác cao như vậy là đủ để khẳng định rằng mô hình có thể được sử dụng một cách đáng tin cậy nhằm hỗ trợ phân loại loài hoa trong môi trường tự nhiên.

Tuy nhiên, cần lưu ý rằng độ chính xác không phải là chỉ số duy nhất để đánh giá hiệu suất mô hình, đặc biệt trong các bài toán mất cân bằng lớp hoặc có chi phí sai lệch cao. Trong các chương liên quan đến xây dựng mô hình, chúng ta sẽ khám phá các kỹ thuật cải thiện hiệu suất

mô hình, cũng như những khía cạnh cần cân nhắc khi điều chỉnh các siêu tham số và lựa chọn thuật toán phù hợp cho từng bài toán cụ thể.

3.3.1.5. Mô hình thứ hai: mô hình cây quyết định

Mô hình **cây quyết định** là một thuật toán học máy thuộc nhóm mô hình **có tham số**, thường được sử dụng trong các bài toán phân loại và hồi quy. Khác với các mô hình dựa trên khoảng cách như k-láng giềng gần nhất, cây quyết định tìm kiếm các quy tắc phân tách dữ liệu dựa trên các ngưỡng của các đặc trưng, từ đó xây dựng một cấu trúc dạng cây nhằm ánh xạ các đặc trưng đầu vào đến biến mục tiêu.

Ý tưởng cốt lõi của cây quyết định là chia nhỏ không gian đặc trưng thành các vùng mà trong mỗi vùng, các mẫu thuộc cùng một lớp mục tiêu là đồng nhất nhất có thể. Mỗi nút trong cây tương ứng với một điều kiện phân chia trên một đặc trưng, và quá trình dự đoán được thực hiện bằng cách dẫn mẫu dữ liệu từ gốc cây đến một lá cây thông qua các điều kiện logic.

Một trong những ưu điểm lớn của cây quyết định là khả năng diễn giải mô hình một cách trực quan, dễ hiểu và không yêu cầu chuẩn hóa dữ liệu đầu vào. Tuy nhiên, cây quyết định đơn giản có thể dễ bị khớp quá mức nếu không được kiểm soát độ sâu hoặc số lượng mẫu tối thiểu tại mỗi nút phân chia.

Trong scikit-learn, mô hình cây quyết định được triển khai trong lớp `DecisionTreeClassifier`, thuộc mô-đun `sklearn.tree`. Giống như các thuật toán học máy khác trong scikit-learn, `DecisionTreeClassifier` là một lớp kế thừa từ `BaseEstimator`, hỗ trợ các phương thức chuẩn như `.fit()`, `.predict()`, và `.score()`.

Để sử dụng mô hình, chúng ta cần khởi tạo một đối tượng từ lớp `DecisionTreeClassifier`. Tại thời điểm khởi tạo, có thể thiết lập nhiều siêu tham số, trong đó quan trọng nhất là `max_depth`, quy định độ sâu tối đa của cây — một yếu tố quan trọng kiểm soát độ phức tạp và khả năng khái quát của mô hình. Trong ví dụ này, chúng ta sẽ giới hạn độ sâu của cây là 2, nghĩa là cây quyết định chỉ có 3 lá:

```
from sklearn.tree import DecisionTreeClassifier

# Khởi tạo mô hình cây quyết định với độ sâu tối đa là 2
tree = DecisionTreeClassifier(max_depth=2)
```

Việc huấn luyện mô hình hoàn toàn tương tự như với thuật toán k-láng giềng gần nhất

```
tree.fit(X_train, Y_train)
```

▼ DecisionTreeClassifier ⓘ ?

```
DecisionTreeClassifier(max_depth=2)
```

Chúng ta dự đoán về loài hoa trên một dữ liệu mới:

```
X_new = np.array([[5, 2.9, 1, 0.2]])  
prediction = tree.predict(X_new)  
print("Dự đoán: {}".format(prediction))
```

Dự đoán: ['setosa']

Xây dựng dự đoán trên toàn bộ dữ liệu kiểm tra:

```
Y_pred = tree.predict(X_test)  
print("Dự đoán tập kiểm tra:\n {}".format(Y_pred))
```

Dự đoán tập kiểm tra:

```
['versicolor' 'virginica' 'setosa' 'versicolor' 'setosa' 'versicolor'  
'versicolor' 'versicolor' 'setosa' 'versicolor' 'versicolor' 'virginica'  
'versicolor' 'setosa' 'setosa' 'virginica' 'versicolor' 'setosa' 'setosa'  
'setosa' 'virginica' 'virginica' 'virginica' 'setosa' 'versicolor'  
'setosa' 'versicolor' 'versicolor' 'versicolor' 'virginica' 'versicolor'  
'versicolor' 'versicolor' 'virginica' 'virginica' 'setosa' 'virginica'  
'virginica' 'virginica' 'virginica' 'setosa' 'setosa' 'versicolor'  
'setosa' 'versicolor']
```

```
print("Điểm tập kiểm tra: {:.2f}".format(knn.score(X_test, Y_test)))
```

Điểm tập kiểm tra: 0.98

Có thể thấy rằng mô hình cây quyết định cũng có khả năng dự đoán chính xác là 98% tương đương với k-láng giềng gần nhất.

3.3.2. Dự đoán giá bất động sản tại Hà Nội

3.3.2.1. Dữ liệu và bài toán dự đoán giá bất động sản

Sau một vài năm đi làm và tích lũy được một khoản tiết kiệm nho nhỏ, bạn bắt đầu nghiêm túc nghĩ đến việc mua một căn nhà để ổn định cuộc sống tại Hà Nội. Tuy nhiên, quá trình tìm hiểu thị trường bất động sản khiến bạn không khỏi bối rối: cùng một diện tích, mức giá có thể chênh lệch đáng kể chỉ vì khác vài con phố; nhiều nơi quảng cáo hấp dẫn nhưng giá lại vượt xa giá trị thực tế; và gần như không có công cụ nào giúp bạn xác định liệu một căn nhà có đang bị định giá quá cao hay không.

Trong bối cảnh thị trường còn chưa minh bạch, thông tin phân tán và thường xuyên bị “thổi giá”, bạn bắt đầu đặt câu hỏi: liệu có thể sử dụng các công cụ học máy để giúp người mua đưa ra quyết định sáng suốt hơn? Nếu có thể thu thập dữ liệu về vị trí, diện tích, loại hình nhà, tiện ích xung quanh, và giá bán thực tế của các bất động sản tương tự, một mô hình học máy hoàn toàn có khả năng học được mối quan hệ giữa các yếu tố này và giá nhà.

Dữ liệu dùng để xây dựng mô hình là dữ liệu về giá của hơn 10,000 ngôi nhà đất tại Hà Nội, bao gồm các thông tin:

- Các thông tin liên quan đến vị trí địa lý của ngôi nhà như địa chỉ, quận, huyện.
- Các thông tin liên quan đến tính chất của ngôi nhà, chẳng hạn như nhà mặt phố hay trong ngõ, diện tích, chiều dài, chiều rộng, số phòng ngủ, số tầng.
- Các thông tin liên quan đến pháp lý của ngôi nhà, chẳng hạn như có sổ đỏ hay chưa.

Mục tiêu của bạn là dựa vào các đặc trưng trên của ngôi nhà để dự đoán mức giá tính theo đơn vị triệu đồng mỗi mét vuông. Dữ liệu được lưu trong file `HN_housing_dataset.csv`. Bạn đọc load dữ liệu như sau:

```
path = 'data/HN_housing_price_raw.csv'
df = pd.read_csv(path, index_col=0, parse_dates=True)
df
```

	Ngày	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	R
1	2021-06-05	phố minh khai, Phường Minh Khai, Quận Hai Bà T...	Quận Hai Bà Trưng	Phường Minh Khai	Nhà ngõ, hẻm	Đã có sổ	4	4 phòng	40 m ²	10 m	
2	2021-06-05	Đường Bồ Đề, Phường Bồ Đề, Quận Long Biên, Hà Nội	Quận Long Biên	Phường Bồ Đề	Nhà ngõ, hẻm	Đã có sổ	5	4 phòng	52 m ²	12 m	4
3	2021-06-05	Đường Tố Hữu, Phường La Khê, Quận Hà Đông, Hà Nội	Quận Hà Đông	Phường La Khê	Nhà mặt phố, mặt tiền	Đã có sổ	5	5 phòng	90 m ²	18 m	
4	2021-06-05	180/61/5, Đường Tây Mỗ, Phường Tây Mỗ, Quận Na...	Quận Nam Từ Liêm	Phường Tây Mỗ	Nhà ngõ, hẻm	Đã có sổ	4	3 phòng	32 m ²	6.6 m	4
7	2021-06-05	Đường Khương Trung, Phường Khương Trung, Quận ...	Quận Thanh Xuân	Phường Khương Trung	Nhà ngõ, hẻm	Đã có sổ	4	4 phòng	38 m ²	10 m	

	Ngày	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	R
...	...	...	...	...	...	...	...	...	...	...	
11466	2021-06-05	101b-c4, ngõ 231, Đường Tân Mai, Phường Tân Ma...	Quận Hoàng Mai	Phường Tân Mai	Nhà ngõ, hẻm	Đã có sổ	2	2 phòng	37 m ²	13 m	
11467	2021-06-05	Đường Nguyễn Trãi, Phường Thượng Đình, Quận Th...	Quận Thanh Xuân	Phường Thượng Đình	Nhà ngõ, hẻm	Đã có sổ	9	nhiều hơn 10 phòng	100 m ²	16 m	
11468	2021-06-05	Đường Hồ Đắc Di, Phường Nam Đồng, Quận Đống Đa...	Quận Đống Đa	Phường Nam Đồng	Nhà mặt phố, mặt tiền	Đã có sổ	5	4 phòng	80 m ²	14 m	
11469	2021-06-05	Đường Khâm Thiên, Phường Khâm Thiên, Quận Đống...	Quận Đống Đa	Phường Khâm Thiên	Nhà mặt phố, mặt tiền	Đã có sổ	3	5 phòng	89 m ²	20 m	
11470	2021-06-05	KĐT Văn Phú, Phường Phú La, Quận Hà	Quận Hà Đông	Phường Phú La	Nhà phố liền kề	Đã có sổ	4	4 phòng	90 m ²	20 m	

	Ngày	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	R
		Đông, Hà Nội									

10980 rows × 12 columns

3.3.2.2. Phân tích khai phá dữ liệu

Không giống như tập dữ liệu *Iris* đã được tiền xử lý và tổ chức sẵn theo một cấu trúc nhất định, dữ liệu giá nhà thường tồn tại dưới dạng thô, chưa được làm sạch hay chuẩn hóa để sẵn sàng cho việc huấn luyện các mô hình học máy. Có thể nhận thấy rằng tập dữ liệu này thường gặp phải nhiều vấn đề phổ biến như giá trị thiếu, định dạng không đồng nhất, sự xuất hiện của các ngoại lệ (outliers), cũng như sự không nhất quán trong đơn vị đo lường giữa các trường dữ liệu.

Đối với những tập dữ liệu như vậy, quá trình phân tích và khai phá dữ liệu (*Exploratory Data Analysis – EDA*) đóng vai trò đặc biệt quan trọng trong chuỗi quy trình xây dựng mô hình học máy. Việc hiểu rõ cấu trúc dữ liệu, phân phối các biến, mối quan hệ giữa các đặc trưng, đồng thời phát hiện và xử lý các vấn đề liên quan đến chất lượng dữ liệu, có ảnh hưởng trực tiếp đến năng lực khái quát hóa và độ chính xác của mô hình đầu ra. Nói cách khác, chất lượng của quá trình EDA là yếu tố then chốt quyết định hiệu suất tổng thể của mô hình học máy được xây dựng từ tập dữ liệu này.

Hơn nữa, quá trình phân tích và khai phá dữ liệu không chỉ đòi hỏi kỹ năng kỹ thuật về xử lý và trực quan hóa dữ liệu, mà còn cần đến kiến thức chuyên sâu về lĩnh vực ứng dụng từ phía người xây dựng mô hình. Đối với bài toán dự đoán giá nhà, điều này có nghĩa là nhà phân tích cần hiểu rõ các yếu tố kinh tế, xã hội, hành chính và hạ tầng có ảnh hưởng đến giá trị bất động sản, cũng như các đặc thù vận hành của thị trường địa phương.

Việc lựa chọn, biến đổi và diễn giải các đặc trưng đầu vào một cách hợp lý — chẳng hạn như vị trí địa lý, tiện ích lân cận, loại hình nhà ở, diện tích sử dụng hay tính pháp lý — không thể chỉ dựa vào các thao tác kỹ thuật thuần túy. Thay vào đó, quá trình này cần được dẫn dắt bởi một sự kết hợp chặt chẽ giữa năng lực phân tích dữ liệu và kiến thức trong lĩnh vực bất động sản.

nhằm đảm bảo rằng mô hình học máy không chỉ chính xác về mặt thống kê, mà còn phản ánh đúng thực tiễn và có khả năng ứng dụng trong môi trường thực tế.

Do đây chỉ là phần giới thiệu về các mô hình học máy, nên chúng tôi chỉ đề xuất một vài kỹ thuật khai phá cho dữ liệu kể trên. Bạn đọc sẽ được tìm hiểu kỹ hơn về các kỹ thuật khai phá dữ liệu trong các phần sau của cuốn sách.

Trước tiên, biến mục tiêu là một biến dạng số thực, nhưng Python đang hiểu đây là một biến dạng chuỗi ký tự do có đơn vị triệu/m², hoặc tỷ/m², hoặc đ/m² phía sau các con số. Để máy tính hiểu được đây là đơn vị con số, chúng ta tạo thêm 2 cột mới, tên là `price_value` chỉ chứa phần số của biến `Giá/mét vuông`, và cột `price_unit` chứa đơn vị của của giá

```
# Tách cột giá nhà thành 2 cột price_value và price_unit
df[["price_value", "price_unit"]] = df["Giá/m2"].str.extract(r"([\d. ,]+\s*(.*)")
```

Trong tập dữ liệu hiện tại, các giá trị thể hiện giá bất động sản được định dạng theo chuẩn số của Việt Nam, trong đó dấu `"."` được sử dụng làm dấu phân tách hàng nghìn và hàng triệu, còn dấu `","` đóng vai trò là dấu phân cách phần thập phân. Ví dụ, giá trị `"2.350.000,5"` biểu thị hai triệu ba trăm năm mươi nghìn đồng và năm hào.

Tuy nhiên, để có thể xử lý và tính toán trong Python, các giá trị này cần được chuyển đổi về kiểu số thực chuẩn quốc tế, trong đó dấu `"."` được sử dụng làm dấu phân tách thập phân và dấu `","` không được chấp nhận trong biểu diễn số. Do đó, cần thực hiện quá trình tiền xử lý bằng cách loại bỏ tất cả các dấu `"."` và chuyển dấu `","` thành `"."` để phù hợp với định dạng số thực trong Python.

Ví dụ, chuỗi `"2.350.000,5"` sau khi chuyển đổi sẽ trở thành số thực `2350000.5`, có thể sử dụng trực tiếp trong các phép tính, mô hình học máy, và các thao tác phân tích dữ liệu khác.

```
df["price_value"] = (
    df["price_value"]
    .astype(str)                # Đảm bảo là chuỗi
    .str.replace(".", "", regex=False)
    .str.replace(",", ".", regex=False)
    .astype(float)              # Chuyển sang số thực
)
```

```
# Xem các đơn vị khác nhau có thể của giá nhà
df["price_unit"].unique()
```

```
array(['triệu/m²', 'tỷ/m²', 'đ/m²'], dtype=object)
```

Trong tập dữ liệu, có thể nhận thấy rằng đơn vị giá được sử dụng không đồng nhất giữa các mẫu quan sát: một số bất động sản được định giá theo đơn vị triệu đồng trên mét vuông, một số khác theo tỷ đồng trên mét vuông, và thậm chí có trường hợp sử dụng đồng trên mét vuông. Nếu không được xử lý thích hợp, sự khác biệt trong đơn vị này có thể dẫn đến việc hiểu sai hoàn toàn về giá trị thực của bất động sản, ảnh hưởng nghiêm trọng đến chất lượng mô hình học máy.

Để đảm bảo tính nhất quán và khả năng so sánh giữa các quan sát, chúng ta thống nhất sử dụng đơn vị triệu đồng trên mét vuông làm chuẩn. Theo đó, tất cả các giá trị thuộc đơn vị khác sẽ được quy đổi về cùng một hệ đo lường bằng các phép biến đổi tương ứng.

```
# Hàm số dùng để chuyển đổi đơn vị
def normalize_price(value, unit):
    if unit == "triệu/m²":
        return value
    elif unit == "đ/m²":
        return value / 10**6
    elif unit == "tỷ/m²":
        return value * 10**3
    else:
        return None # hoặc np.nan nếu đơn vị không rõ

# Áp dụng cho dữ liệu giá nhà
df["price_value"] = df.apply(
    lambda row: normalize_price(row["price_value"], row["price_unit"]), axis=1
)
```

```
# Xem lại các mô tả thống kê cho biến price_value
df["price_value"].describe()
```

```
count    10980.000000
mean      109.191818
std       139.377876
min        0.054081
25%       72.500000
50%       91.180000
75%      117.310000
max      7894.736842
Name: price_value, dtype: float64
```

Có thể nhận thấy rằng một số bất động sản trong tập dữ liệu đã bị nhập sai đơn vị đo lường giá, dẫn đến những giá trị bất hợp lý. Ví dụ, bất động sản có mức giá thấp nhất được ghi nhận là 0.05 triệu đồng trên mỗi mét vuông – một con số không thực tế, nhiều khả năng xuất phát từ việc nhầm lẫn đơn vị từ triệu đồng hoặc tỷ đồng sang đồng. Ngược lại, bất động sản có mức giá cao nhất được ghi nhận là 7.894 triệu đồng trên mỗi mét vuông, tương đương 7,9 tỷ đồng/m², cũng là một giá trị vượt xa giới hạn hợp lý và có thể do sai sót trong nhập liệu.

Trong các chương tiếp theo của cuốn sách, những quan sát như vậy sẽ được phân loại là **giá trị ngoại lai**, hay *outliers*. Nếu không được xử lý thích hợp, các giá trị ngoại lai này có thể ảnh hưởng nghiêm trọng đến hiệu suất và độ tin cậy của các mô hình phân tích dữ liệu, gây sai lệch trong kết quả và làm giảm khả năng tổng quát hóa của mô hình đối với dữ liệu thực tế.

Quy trình xử lý dữ liệu sẽ được thực hiện tuần tự đối với biến mục tiêu cũng như từng biến đặc trưng, cho đến khi toàn bộ các biến trong mô hình được chuẩn hóa đầy đủ về kiểu dữ liệu và giá trị. Việc đảm bảo tính nhất quán và hợp lệ của các biến là điều kiện tiên quyết để xây dựng các mô hình phân tích có độ chính xác và khả năng tổng quát hóa cao. Toàn bộ quá trình này sẽ được trình bày chi tiết thông qua các phần thực hành trong các chương sau của cuốn sách.

Dữ liệu sau khi được làm sạch bằng các kỹ thuật được trình bày trong các chương liên quan đến tiền xử lý và xử lý dữ liệu được lưu trong file `HN_housing_price_clean.csv`

```
path = 'data/HN_housing_price_clean.csv'
df = pd.read_csv(path, index_col=0, parse_dates=True)
df
```

	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng	C
1	phố minh khai, Phường Minh Khai, Quận Hai Bà T...	Quận Hai Bà Trưng	Phường Minh Khai	Nhà ngõ, hẻm	Đã có sổ	4	4	40.0	10.0	4.0	
2	Đường Bồ Đề, Phường Bồ Đề, Quận Long Biên, Hà Nội	Quận Long Biên	Phường Bồ Đề	Nhà ngõ, hẻm	Đã có sổ	5	4	50.4	12.0	4.2	
3	Đường Tố Hữu, Phường La Khê, Quận Hà Đông, Hà Nội	Quận Hà Đông	Phường La Khê	Nhà mặt phố, mặt tiền	Đã có sổ	5	5	90.0	18.0	5.0	
4	180/61/5, Đường Tây Mỗ, Phường Tây Mỗ, Quận Na...	Quận Nam Từ Liêm	Phường Tây Mỗ	Nhà ngõ, hẻm	Đã có sổ	4	3	29.7	6.6	4.5	
7	Đường Khương Trung, Phường Khương Trung, Quận ...	Quận Thanh Xuân	Phường Khương Trung	Nhà ngõ, hẻm	Đã có sổ	4	4	30.0	10.0	3.0	

	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng	C
...	...	...	...	...	...	...	...	...	...	...	
11466	101b-c4, ngõ 231, Đường Tân Mai, Phường Tân Mai...	Quận Hoàng Mai	Phường Tân Mai	Nhà ngõ, hẻm	Đã có sổ	2	2	39.0	13.0	3.0	
11467	Đường Nguyễn Trãi, Phường Thượng Đình, Quận Th...	Quận Thanh Xuân	Phường Thượng Đình	Nhà ngõ, hẻm	Đã có sổ	9	10+	96.0	16.0	6.0	
11468	Đường Hồ Đắc Di, Phường Nam Đồng, Quận Đống Đa...	Quận Đống Đa	Phường Nam Đồng	Nhà mặt phố, mặt tiền	Đã có sổ	5	4	70.0	14.0	5.0	
11469	Đường Khâm Thiên, Phường Khâm Thiên, Quận Đống...	Quận Đống Đa	Phường Khâm Thiên	Nhà mặt phố, mặt tiền	Đã có sổ	3	5	80.0	20.0	4.0	
11470	KĐT Văn Phú, Phường Phú La, Quận Hà	Quận Hà Đông	Phường Phú La	Nhà phố liền kề	Đã có sổ	4	4	80.0	20.0	4.0	

	Địa chỉ	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng	C
	Đông, Hà Nội										

10701 rows × 11 columns

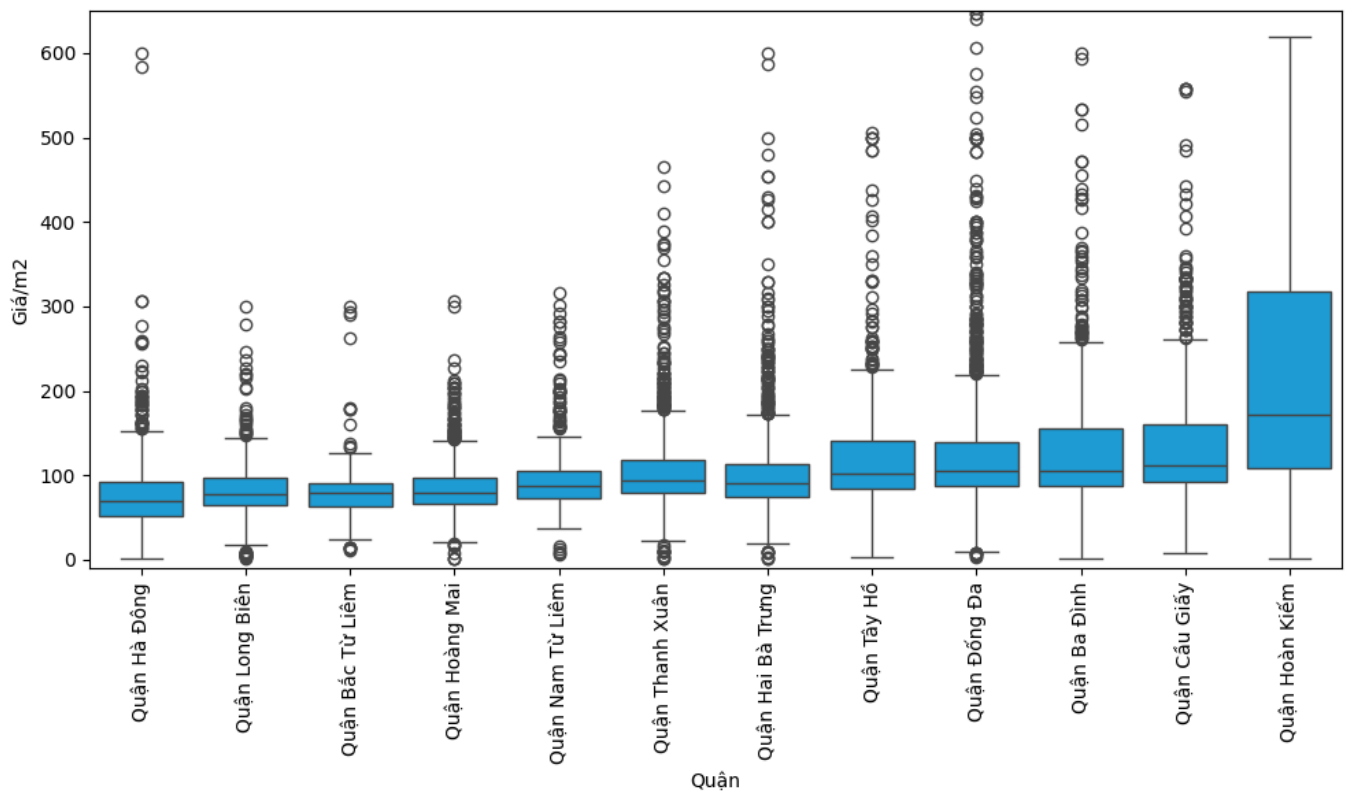
Một yêu cầu then chốt trong quá trình phân tích và khai phá dữ liệu là xác định các đặc trưng sẵn có hoặc xây dựng thêm các đặc trưng mới có ý nghĩa, nhằm nâng cao hiệu quả của mô hình dự đoán – chẳng hạn như mô hình dự đoán giá bất động sản. Đối với các biến phân loại, việc đánh giá xem liệu một biến có ảnh hưởng đến biến mục tiêu hay không thường được thực hiện thông qua kết hợp giữa phương pháp trực quan hóa, chẳng hạn như biểu đồ hộp, và kiểm định thống kê.

Chẳng hạn, để kiểm tra xem giá nhà có sự khác biệt đáng kể giữa các quận hay không, ta có thể tiến hành trực quan hóa như sau:

```
df1 = df.copy()

# Sắp xếp các quận theo giá trung bình
df1['Quận'] = pd.Categorical(df1['Quận'],
                             categories=df1.groupby('Quận')['Giá/m2'].mean().sort_values(
                                 ordered=True))

# Vẽ boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x='Quận', y='Giá/m2', data=df1, color = "#00A7F0")
plt.xticks(rotation=90)
plt.ylim(-10, 650) # Giới hạn trục y
plt.tight_layout()
plt.show()
```

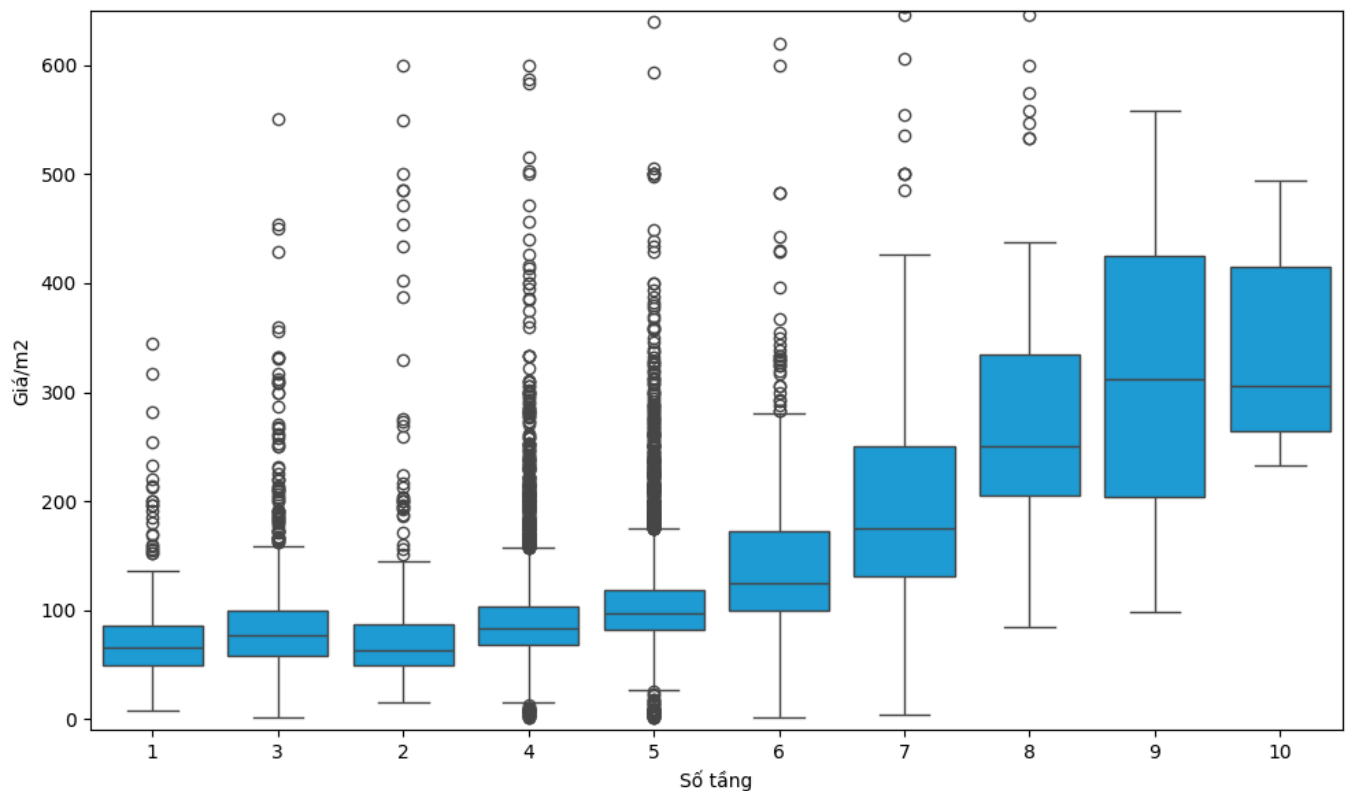


Kết quả trực quan hóa cho thấy giá nhà thực sự có sự khác biệt rõ rệt giữa các quận. Đáng chú ý, quận Hoàn Kiếm có mức giá nhà trung bình cao vượt trội so với các quận còn lại. Trong khi đó, quận Tây Hồ – mặc dù theo nhận định ban đầu được cho là có mức giá cao – trên thực tế chỉ tương đương với các quận như Ba Đình hoặc Đống Đa về mặt giá trung bình.

Bên cạnh vị trí địa lý, một yếu tố khác cũng có khả năng ảnh hưởng đáng kể đến giá nhà là số tầng của ngôi nhà. Do số tầng là một biến phân loại có thứ tự (ordinal categorical variable), biểu đồ hộp (boxplot) tiếp tục là công cụ hữu ích để khám phá mối quan hệ giữa đặc trưng này và biến mục tiêu.

```
# Sắp xếp các quận theo giá trung bình
df1 = df
df1['Số tầng'] = pd.Categorical(df1['Số tầng'],
                                categories=df1.groupby('Số tầng')['Giá/m²'].mean().sort_values(
                                    ascending=True).index,
                                ordered=True)

# Vẽ boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x='Số tầng', y='Giá/m²', data=df1, color = "#00A7F0")
plt.ylim(-10, 650) # Giới hạn trục y
plt.tight_layout()
plt.show()
```



Kết quả phân tích cho thấy mối quan hệ thuận chiều giữa số tầng của một ngôi nhà và mức giá tính trên mỗi mét vuông, phù hợp với giả thuyết được đặt ra từ trước. Diễn giải theo cách đơn giản, giá trị của một bất động sản không chỉ phản ánh giá trị của quyền sử dụng đất, mà còn bao gồm chi phí xây dựng. Khi số tầng tăng, chi phí xây dựng trên mỗi mét vuông sàn cũng có xu hướng tăng theo, do các yêu cầu kỹ thuật, kết cấu và vật liệu cao tầng phức tạp hơn. Do đó, giá nhà trên mỗi mét vuông ở những ngôi nhà có nhiều tầng thường cao hơn so với nhà thấp tầng.

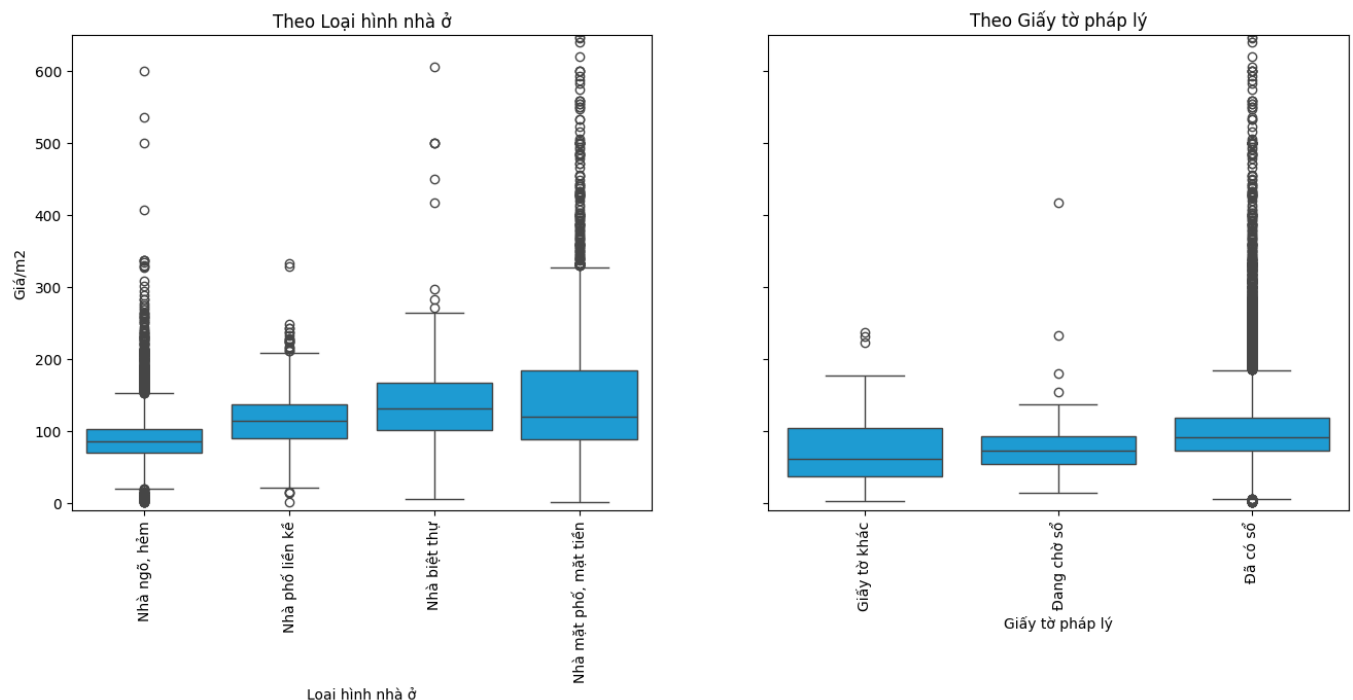
Chúng ta tiếp tục với hai biến **Loại hình nhà ở** và **Giấy tờ pháp lý**

```
# Tạo figure và 2 axes nằm ngang cạnh nhau
fig, axes = plt.subplots(1, 2, figsize=(16, 6), sharey=True)

# ----- Biểu đồ 1: theo Loại hình nhà ở -----
# Sắp xếp theo giá trung bình
df['Loại hình nhà ở'] = pd.Categorical(
    df['Loại hình nhà ở'],
    categories=df.groupby('Loại hình nhà ở')['Giá/m2'].mean().sort_values().index,
    ordered=True
)
sns.boxplot(x='Loại hình nhà ở', y='Giá/m2', data=df, ax=axes[0], color = "#00A7F0")
plt.ylim(-10, 650) # Giới hạn trục y
axes[0].set_title('Theo Loại hình nhà ở')
axes[0].tick_params(axis='x', rotation=90)

# ----- Biểu đồ 2: theo Giấy tờ pháp lý -----
df['Giấy tờ pháp lý'] = pd.Categorical(
    df['Giấy tờ pháp lý'],
    categories=df.groupby('Giấy tờ pháp lý')['Giá/m2'].mean().sort_values().index,
    ordered=True
)
sns.boxplot(x='Giấy tờ pháp lý', y='Giá/m2', data=df, ax=axes[1], color = "#00A7F0")
plt.ylim(-10, 650) # Giới hạn trục y
axes[1].set_title('Theo Giấy tờ pháp lý')
axes[1].tick_params(axis='x', rotation=90)

plt.show()
```



Phân tích dữ liệu cho thấy giá nhà đối với các loại hình như biệt thự và nhà mặt phố có xu hướng cao hơn rõ rệt so với các loại hình như nhà phố liền kề hay nhà trong ngõ hẻm. Khi xét

đến yếu tố pháp lý, những bất động sản đã có giấy chứng nhận quyền sử dụng đất (sổ đỏ) thường có mức giá cao hơn so với các trường hợp chưa có sổ. Ngoài ra, các ngôi nhà đang trong quá trình chờ cấp sổ đỏ cũng có mức giá trung bình nhỉnh hơn một chút so với nhóm nhà có “giấy tờ khác” – một nhóm thường kém minh bạch về mặt pháp lý.

Đối với các biến định lượng (kiểu số liên tục), phương pháp trực quan hóa phổ biến là sử dụng đồ thị phân tán để mô tả mối quan hệ giữa biến giải thích và biến mục tiêu. Chẳng hạn, để kiểm tra xem diện tích đất có liên quan đến giá nhà tính trên mỗi mét vuông hay không, ta có thể sử dụng biểu đồ phân tán như sau:

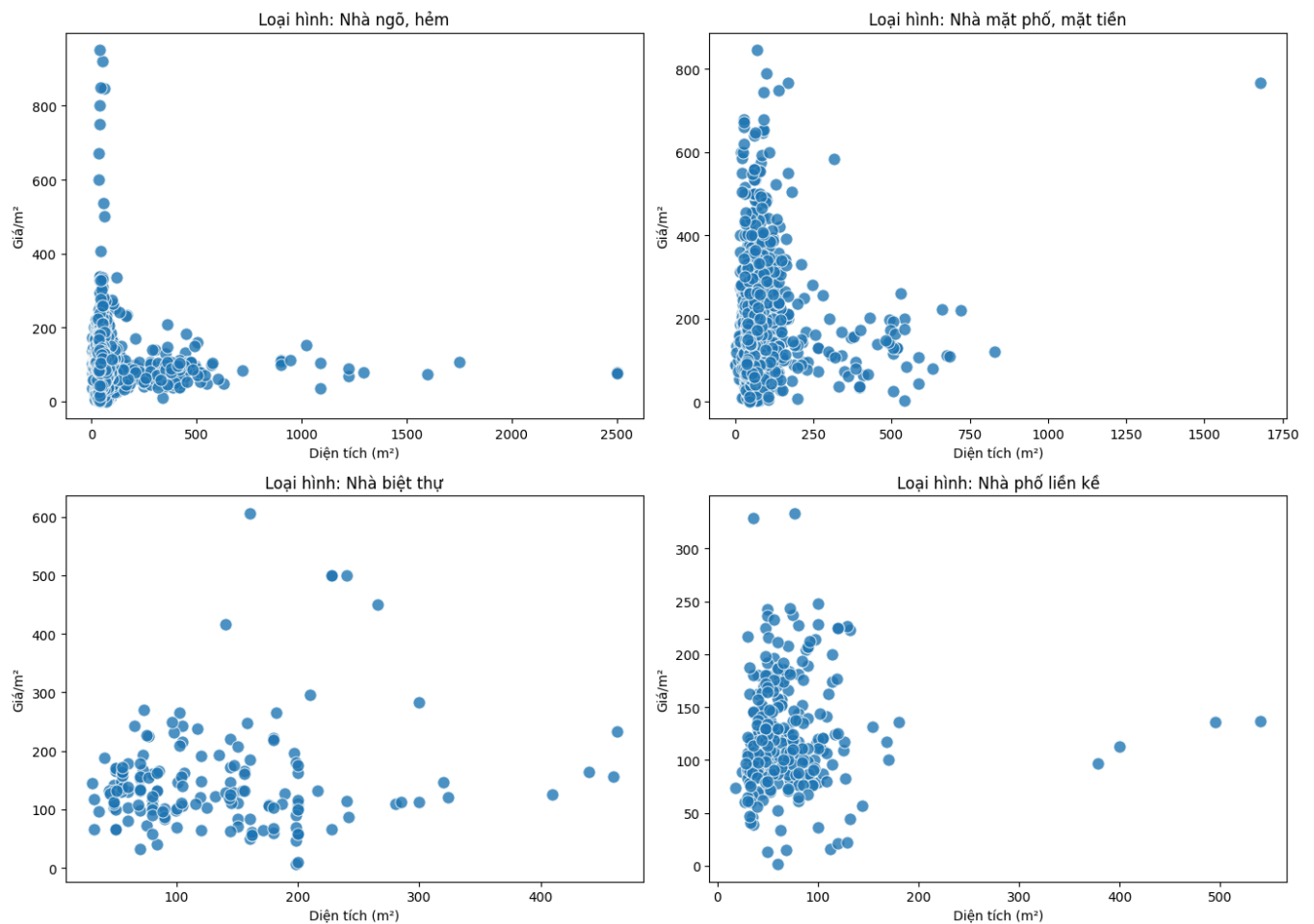
```
# Xác định các giá trị duy nhất của biến "Loại hình nhà ở"
loai_hinh = df['Loại hình nhà ở'].dropna().unique()

# Tạo figure 2x2
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten() # Chuyển thành 1D list để dễ duyệt

# Vẽ từng scatter plot cho từng loại hình
for i, loai in enumerate(loai_hinh):
    subset = df[df['Loại hình nhà ở'] == loai]
    sns.scatterplot(x='Diện tích', y='Giá/m2', data=subset, ax=axes[i], s = 90, alpha
axes[i].set_title(f'Loại hình: {loai}')
axes[i].set_xlabel('Diện tích (m²)')
axes[i].set_ylabel('Giá/m²')

# Xóa subplot dư nếu ít hơn 4 loại hình
for j in range(len(loai_hinh), 4):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```



Trái với giả định ban đầu cho rằng diện tích đất của một ngôi nhà có thể ảnh hưởng đến giá bán tính trên mỗi mét vuông, biểu đồ phân tán lại không cho thấy mối quan hệ rõ ràng giữa hai biến này. Điều này cho thấy rằng, trong một số trường hợp, những đặc trưng tương chừng có ý nghĩa lại không thực sự đóng vai trò quyết định đối với biến mục tiêu.

Quá trình phân tích mối quan hệ giữa các biến đặc trưng và biến mục tiêu sẽ tiếp tục được thực hiện theo hướng tiếp cận tương tự, nhằm lựa chọn ra tập biến đầu vào phù hợp cho mô hình dự đoán. Ngoài việc sử dụng các biến sẵn có trong dữ liệu, người phân tích hoàn toàn có thể xây dựng thêm các biến đặc trưng mới có khả năng cải thiện hiệu quả dự báo. Ví dụ, từ địa chỉ chi tiết của bất động sản, ta có thể tính toán một số đại lượng được giả định là có ảnh hưởng đáng kể đến giá nhà như: khoảng cách đến trục đường chính, mức độ kết nối với hệ thống đường cao tốc, khoảng cách đến trung tâm thương mại, cơ sở y tế, trường học, v.v.

Các phương pháp tạo biến đặc trưng nâng cao như vậy sẽ được trình bày chi tiết trong các phần thực hành tiếp theo của cuốn sách.

3.3.2.3. Xây dựng mô hình dự đoán giá nhà

Đối với biến mục tiêu dạng số, các mô hình dự đoán được gọi là các **mô hình hồi quy**, hay *regression*. Chúng ta sẽ xây dựng một mô hình **rừng ngẫu nhiên**, hay *Random Forest*, nhằm dự đoán giá nhà trên mỗi mét vuông. Rừng ngẫu nhiên là một thuật toán học có giám sát thuộc nhóm mô hình tổ hợp *ensemble learning*, hoạt động bằng cách xây dựng nhiều cây quyết định độc lập trên các tập dữ liệu con được lấy mẫu ngẫu nhiên từ tập huấn luyện. Đối với bài toán hồi quy, dự đoán cuối cùng của mô hình rừng ngẫu nhiên được xác định bằng cách tổng hợp bằng trung bình các dự đoán từ các cây riêng lẻ. Cách tiếp cận như vậy giúp mô hình có khả năng tổng quát hóa tốt hơn và giảm thiểu hiện tượng khớp quá mức của mô hình.

Tương tự như ví dụ trước, tập dữ liệu sẽ được chia thành hai phần: tập huấn luyện và tập kiểm tra. Tập huấn luyện được sử dụng để xây dựng mô hình, trong khi tập kiểm tra giúp đánh giá khả năng dự đoán của mô hình trên dữ liệu chưa từng được nhìn thấy.

```
# Thống nhất các biến kiểu số
num_vars = ["Diện tích", "Dài", "Rộng", "Giá/m2"]
for var in num_vars:
    df[var] = df[var].astype('float')
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    df[["Quận", "Huyện", "Loại hình nhà ở", "Giấy tờ pháp lý", "Số tầng", "Số phòng ngủ"],
    random_state = 0, test_size=0.3)
```

```
X_train
```


	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng
911	Quận Hoàng Mai	Phường Tân Mai	Nhà mặt phố, mặt tiền	Đã có sổ	5	3	37.4	11.0	3.4
2433	Quận Long Biên	Phường Long Biên	Nhà mặt phố, mặt tiền	Đã có sổ	5	4	35.0	7.0	5.0
7596	Quận Cầu Giấy	Phường Mai Dịch	Nhà ngõ, hẻm	Đã có sổ	5	5	40.0	10.0	4.0
9220	Quận Thanh Xuân	Phường Hạ Đình	Nhà mặt phố, mặt tiền	Đã có sổ	4	2	27.0	9.0	3.0
918	Quận Hà Đông	Phường Phú Lương	Nhà ngõ, hẻm	Đã có sổ	4	3	35.0	10.0	3.5
...	...	...	...	...	...	...	...	...	...
9888	Quận Ba Đình	Phường Giảng Võ	Nhà ngõ, hẻm	Đã có sổ	4	3	40.0	8.0	5.0
5166	Quận Tây Hồ	Phường Quảng An	Nhà ngõ, hẻm	Đã có sổ	5	4	30.0	10.0	3.0
3476	Quận Ba Đình	Phường Cống Vị	Nhà mặt phố, mặt tiền	Đã có sổ	5	7	50.0	10.0	5.0

	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng
10553	Quận Hoàng Mai	Phường Lĩnh Nam	Nhà ngỗ, hẻm	Đã có sổ	3	3	33.0	11.0	3.0
2912	Quận Thanh Xuân	Phường Khương Trung	Nhà mặt phố, mặt tiền	Đã có sổ	4	4	100.0	25.0	4.0

7490 rows × 9 columns

```
X_test.shape
```

```
(3211, 9)
```

Tổng cộng có 7.490 bất động sản được sử dụng làm tập dữ liệu huấn luyện để xây dựng mô hình và 3.211 bất động sản còn lại được sử dụng làm tập kiểm tra nhằm đánh giá hiệu quả dự đoán của mô hình.

Trước khi đưa các biến vào mô hình học máy, một bước tiền xử lý quan trọng là chuyển đổi các biến phân loại sang dạng **biến giả**, hay *biến kiểu one-hot* nhằm đảm bảo tính tương thích với các thuật toán mô hình hóa.

```
# Chuyển đổi biến sang kiểu one hot
X_train_encoded = pd.get_dummies(X_train, drop_first=True)
X_test_encoded = pd.get_dummies(X_test, drop_first=True)

# Đồng bộ hóa cột giữa train/test
X_train_encoded, X_test_encoded = X_train_encoded.align(X_test_encoded, join='left', axis=1)
```

Sau khi hoàn tất quá trình tiền xử lý dữ liệu, mô hình rừng ngẫu nhiên sẽ được gọi từ thư viện `sklearn` để tiến hành huấn luyện trên tập dữ liệu đã chuẩn hóa:

```
from sklearn.ensemble import RandomForestRegressor

# Khởi tạo mô hình Random Forest
rf_model = RandomForestRegressor(
    n_estimators = 2000,      # số lượng cây
    max_depth = 50,          # độ sâu tối đa
    max_features = 9,        # số lượng đặc trưng của mỗi cây quyết định
    random_state = 10,       # khởi tạo ngẫu nhiên để tái lập kết quả
)

# Huấn luyện mô hình với dữ liệu huấn luyện
rf_model.fit(X_train_encoded, Y_train)
```

Sau khi mô hình rừng ngẫu nhiên đã được huấn luyện và được lưu dưới tên `rf_model`, chúng ta có thể sử dụng mô hình này để dự đoán giá bất động sản trên tập dữ liệu kiểm tra `X_test`.

Việc dự đoán được thực hiện thông qua phương thức `.predict()` của mô hình đã huấn luyện.

```
Y_pred = rf_model.predict(X_test_encoded)
```

Chúng ta có thể so sánh giá của 10 ngôi nhà đầu tiên trong dữ liệu kiểm tra và giá được dự đoán từ mô hình rừng ngẫu nhiên

```
X_test[:10]
```

	Quận	Huyện	Loại hình nhà ở	Giấy tờ pháp lý	Số tầng	Số phòng ngủ	Diện tích	Dài	Rộng
6058	Quận Đống Đa	Phường Ô Chợ Dừa	Nhà mặt phố, mặt tiền	Đã có sổ	4	4	45.00	15.0	3.0
3083	Quận Hà Đông	Phường Phúc La	Nhà mặt phố, mặt tiền	Đã có sổ	4	4	40.00	10.0	4.0
1218	Quận Cầu Giấy	Phường Mai Dịch	Nhà ngõ, hẻm	Đã có sổ	4	6	57.96	12.6	4.6
131	Quận Nam Từ Liêm	Phường Phương Canh	Nhà ngõ, hẻm	Đã có sổ	4	3	31.20	7.8	4.0
11129	Quận Hai Bà Trưng	Phường Minh Khai	Nhà mặt phố, mặt tiền	Đã có sổ	4	4	45.00	9.0	5.0
6197	Quận Bắc Từ Liêm	Phường Phú Diễn	Nhà ngõ, hẻm	Giấy tờ khác	4	4	30.00	10.0	3.0
4267	Quận Long Biên	Phường Ngọc Thụy	Nhà ngõ, hẻm	Đã có sổ	5	3	27.00	9.0	3.0
5824	Quận Đống Đa	Phường Ô Chợ Dừa	Nhà ngõ, hẻm	Đã có sổ	4	4	48.00	12.0	4.0
8529	Quận Hà Đông	Phường Phúc La	Nhà biệt thự	Đã có sổ	5	4	80.00	16.0	5.0
5842	Quận Đống Đa	Phường Láng Hạ	Nhà ngõ, hẻm	Đã có sổ	6	3	32.00	8.0	4.0

```
pd.DataFrame({
    'Giá thực tế': Y_test[:10],
    'Giá dự đoán': Y_pred[:10]})
```

	Giá thực tế	Giá dự đoán
6058	270.00	182.666425
3083	125.00	96.177279
1218	127.59	112.345545
131	79.03	77.220001
11129	162.86	105.353566
6197	78.57	74.992164
4267	98.33	81.359276
5824	100.00	131.327675
8529	100.00	98.833940
5842	120.00	118.905283

Kết quả dự đoán từ mô hình cho thấy phần nào khả năng phản ánh sự biến động của giá bất động sản trên thị trường.

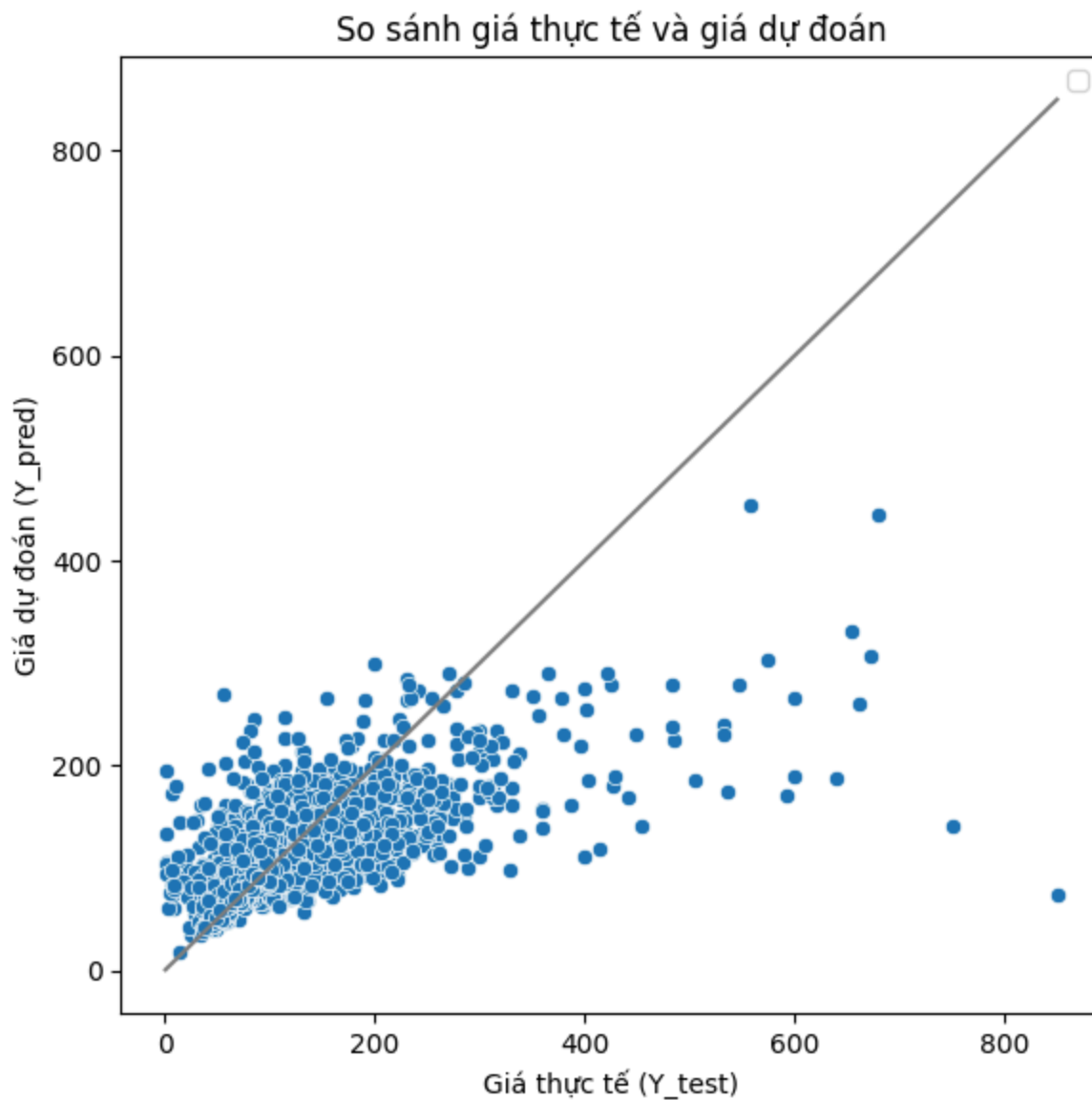
- Một số trường hợp có mức sai lệch nhỏ giữa giá trị thực tế và giá trị dự đoán, cho thấy mô hình hoạt động hiệu quả đối với những quan sát điển hình — chẳng hạn như các bất động sản mang chỉ số 131, 8529 hoặc 5842.
- Tuy nhiên, vẫn tồn tại những trường hợp mô hình chưa dự đoán chính xác, đặc biệt là các bất động sản có đặc điểm không phổ biến hoặc khác biệt đáng kể, như các trường hợp mang chỉ số 6058 và 11129, nơi giá trị dự đoán thấp hơn nhiều so với thực tế.

Kết quả dự đoán và giá trị thực tế của 3.211 bất động sản trong tập dữ liệu kiểm tra được minh họa thông qua đồ thị dưới đây. Hình vẽ cho phép trực quan hóa mức độ phù hợp giữa mô hình dự đoán và dữ liệu thực, đồng thời hỗ trợ nhận diện các điểm sai lệch lớn hoặc các xu hướng sai khác có hệ thống.

```
# Tạo đồ thị scatter: Giá thực tế vs Giá dự đoán
plt.figure(figsize=(6, 6))
sns.scatterplot(x=Y_test, y=Y_pred)

# Thêm đường chéo y = x để tham chiếu
plt.plot([Y_test.min(), Y_test.max()],
         [Y_test.min(), Y_test.max()], color = "grey")

# Nhãn trục và tiêu đề
plt.xlabel('Giá thực tế (Y_test)')
plt.ylabel('Giá dự đoán (Y_pred)')
plt.title('So sánh giá thực tế và giá dự đoán')
plt.legend()
plt.tight_layout()
plt.show()
```



Trong các bài toán hồi quy, hiệu quả của mô hình thường được đánh giá thông qua ba thước đo phổ biến:

- Sai số tuyệt đối trung bình, hay *Mean Absolute Error*, đo lường sai lệch trung bình tuyệt đối giữa giá trị dự đoán và giá trị thực tế.
- Sai số bình phương trung bình, hay *Mean Squared Error*, tính trung bình của bình phương sai số, nhấn mạnh các sai lệch lớn.
- Hệ số xác định, hay R-squared, phản ánh tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình, dao động từ 0 đến 1.

Các thước đo này cung cấp cái nhìn toàn diện về độ chính xác và độ tin cậy của mô hình hồi quy trong việc dự đoán dữ liệu thực tế. Các sai số này được tích hợp sẵn trong thư viện

`sklearn`

```
from sklearn.metrics import root_mean_squared_error, r2_score, mean_absolute_error  
mean_absolute_error(Y_test, Y_pred)
```

28.038014384108962

```
r2_score(Y_test, Y_pred)
```

0.44714869744024754

Kết quả đánh giá cho thấy chỉ số R-squared đạt giá trị 0.447, điều này đồng nghĩa với việc mô hình hiện tại có khả năng giải thích khoảng 44,7% sự biến động của giá bất động sản dựa trên các đặc trưng đầu vào.

Bên cạnh đó, sai số tuyệt đối trung bình là 28.0, cho thấy rằng trung bình mô hình dự đoán sai lệch khoảng 28 triệu đồng mỗi mét vuông so với giá trị thực tế. Mức sai số này phản ánh mức độ chênh lệch cần được cải thiện trong các bước tinh chỉnh tiếp theo của mô hình.

Mặc dù mô hình hiện tại đã phần nào phản ánh được ảnh hưởng của các đặc trưng đầu vào đến giá bất động sản, nhưng độ chính xác vẫn chưa đạt kỳ vọng. Để cải thiện năng lực dự đoán của mô hình, có thể xem xét các hướng tiếp cận sau:

1. Tiếp tục chuẩn hóa và biến đổi dữ liệu hiện có: với các biến phân loại, một kỹ thuật hiệu quả là gộp các nhóm có giá trị trung bình của biến mục tiêu tương đương nhằm giảm nhiễu và cải thiện tính khái quát của mô hình.

2. Tạo thêm các biến mới có ý nghĩa kinh tế: việc thiết kế các đặc trưng mới có thể cải thiện đáng kể hiệu năng dự đoán. Ví dụ, với hai bất động sản có cùng diện tích, bất động sản có mặt tiền rộng thường có giá cao hơn, điều này gợi ý có thể bổ sung biến như tỷ lệ mặt tiền trên chiều sâu (Rộng/Dài) để đưa thêm thông tin về hình dạng lô đất vào mô hình.
3. Tích hợp dữ liệu từ các nguồn bên ngoài: một số yếu tố quan trọng như vị trí địa lý, hạ tầng, và tiện ích xung quanh không có sẵn trong dữ liệu gốc nhưng có thể thu thập từ các hệ thống bản đồ thông qua biến địa chỉ của bất động sản. Các đặc trưng như khoảng cách đến đường lớn, độ rộng mặt đường, khoảng cách đến trường học, bệnh viện, trung tâm thương mại, chợ... có thể đóng vai trò then chốt trong việc cải thiện khả năng dự báo.
4. Xem xét các lớp mô hình học máy khác: khi bộ đặc trưng đã đầy đủ và có chất lượng, nên thử nghiệm với nhiều thuật toán mô hình khác nhau, không giới hạn trong rừng ngẫu nhiên. Trên thực tế, các mô hình thuộc nhóm học tăng cường, hay boosting, như **Gradient Boosting Machines (GBM)**, **XGBoost**, **LightGBM** thường cho hiệu năng cao hơn trong các bài toán dự đoán giá bất động sản.

3.3.3. Tìm khách hàng tiềm năng cho chiến dịch quảng cáo

3.3.3.1. Giới thiệu về dữ liệu

Trong bối cảnh cạnh tranh gay gắt giữa các tổ chức tài chính, việc tối ưu hóa chiến dịch tiếp thị nhằm tiếp cận đúng tệp khách hàng tiềm năng là yếu tố then chốt giúp tăng tỷ lệ chuyển đổi, giảm chi phí, và nâng cao hiệu quả hoạt động.

Là một chuyên gia phân tích dữ liệu trong một ngân hàng, bạn được giao phụ trách một chiến dịch tiếp thị đến khách hàng về mở sổ tiết kiệm có kỳ hạn. Bạn được trung tâm dữ liệu gửi cho thông tin của 20 nghìn khách hàng tiềm năng, nhưng với mức kinh phí được giao, bạn chỉ có thể tiếp cận đến 10% số khách hàng trong danh sách, nghĩa là khoảng 2000 người. Một cách tự nhiên, câu hỏi đặt ra với bạn là: *"Nên tiếp cận 2000 khách hàng nào sao cho tổng số khách hàng đồng ý mở sổ tiết kiệm kỳ hạn?"*

Với kinh nghiệm làm việc với dữ liệu, bạn hiểu rằng phân tích dữ liệu lịch sử từ các chiến dịch tiếp thị trước đây, bạn có thể xây dựng một mô hình dự đoán, giúp tính toán được khả năng

đồng ý mở sổ tiết kiệm của khách hàng. Tiếp cận đến 2000 khách hàng có khả năng đồng ý cao nhất sẽ giúp bạn ưu tiên gọi điện đến đúng đối tượng và tối ưu được hiệu quả tiếp thị.

Tập dữ liệu thu thập từ các chiến dịch tiếp thị qua điện thoại của một ngân hàng tại Việt Nam. Mỗi dòng trong dữ liệu đại diện cho một khách hàng đã được gọi điện trong một chiến dịch cụ thể. Tập dữ liệu bao gồm:

- Biến mục tiêu là biến `term_deposit`, cho biết khách hàng có đăng ký tiền gửi có kỳ hạn hay không? Đây là một biến nhị phân chỉ nhận giá trị 0 nếu khách hàng không đồng ý và 1 nếu khách hàng đồng ý.
- Các biến liên quan đến thông tin về khách hàng được mô tả trong bảng dưới đây:

Tên biến	Diễn giải	Kiểu dữ liệu
<code>age</code>	Tuổi của khách hàng	Số
<code>job</code>	Nghề nghiệp (VD: admin., technician, retired, ...)	Phân loại
<code>marital</code>	Tình trạng hôn nhân	Phân loại
<code>education</code>	Trình độ học vấn	Phân loại
<code>default</code>	Đang nợ xấu không?	Phân loại
<code>balance</code>	Tổng số tiền gửi và dư nợ của khách hàng?	Số (triệu đồng)
<code>housing</code>	Có đang vay mua nhà không?	Phân loại
<code>loan</code>	Có đang vay tiêu dùng không?	Phân loại

- Các biến liên quan đến thông tin về cuộc gọi gần nhất

Tên biến	Diễn giải	Kiểu dữ liệu
<code>month</code>	Tháng gọi điện	Phân loại
<code>dayofweek</code>	Thứ trong tuần gọi điện	Phân loại

- Các biến liên quan đến thông tin từ các chiến dịch tiếp thị

Tên biến	Diễn giải	Kiểu dữ liệu
<code>campaign</code>	Số lần gọi trong chiến dịch hiện tại	Số
<code>pdays</code>	Số ngày kể từ lần liên hệ trước đó (999 nếu chưa từng liên hệ)	Số
<code>previous</code>	Số lần liên hệ trong các chiến dịch trước	Số

Dữ liệu được lưu trong file `BankCustomerData.csv`. Chúng ta đọc dữ liệu vào Python như sau:

```
path = 'data/BankCustomerData.csv'
df = pd.read_csv(path, index_col=0, parse_dates=True)
df
```

	age	job	marital	education	default	balance	housing	loan	data
ID									
1	58	management	2	tertiary	0	2143	1	0	
2	44	technician	1	secondary	0	29	1	0	
3	33	entrepreneur	2	secondary	0	2	1	1	
4	47	blue-collar	2	unknown	0	1506	1	0	
5	33	unknown	1	unknown	0	1	0	0	
...	...	...	...	...	...	...	...	...	
42635	21	student	1	secondary	0	2488	0	0	1
42636	87	retired	2	primary	0	2190	0	0	1
42637	34	blue-collar	2	primary	0	6718	0	0	1
42638	22	student	1	secondary	0	254	0	0	1
42639	32	management	1	tertiary	0	1962	0	0	1

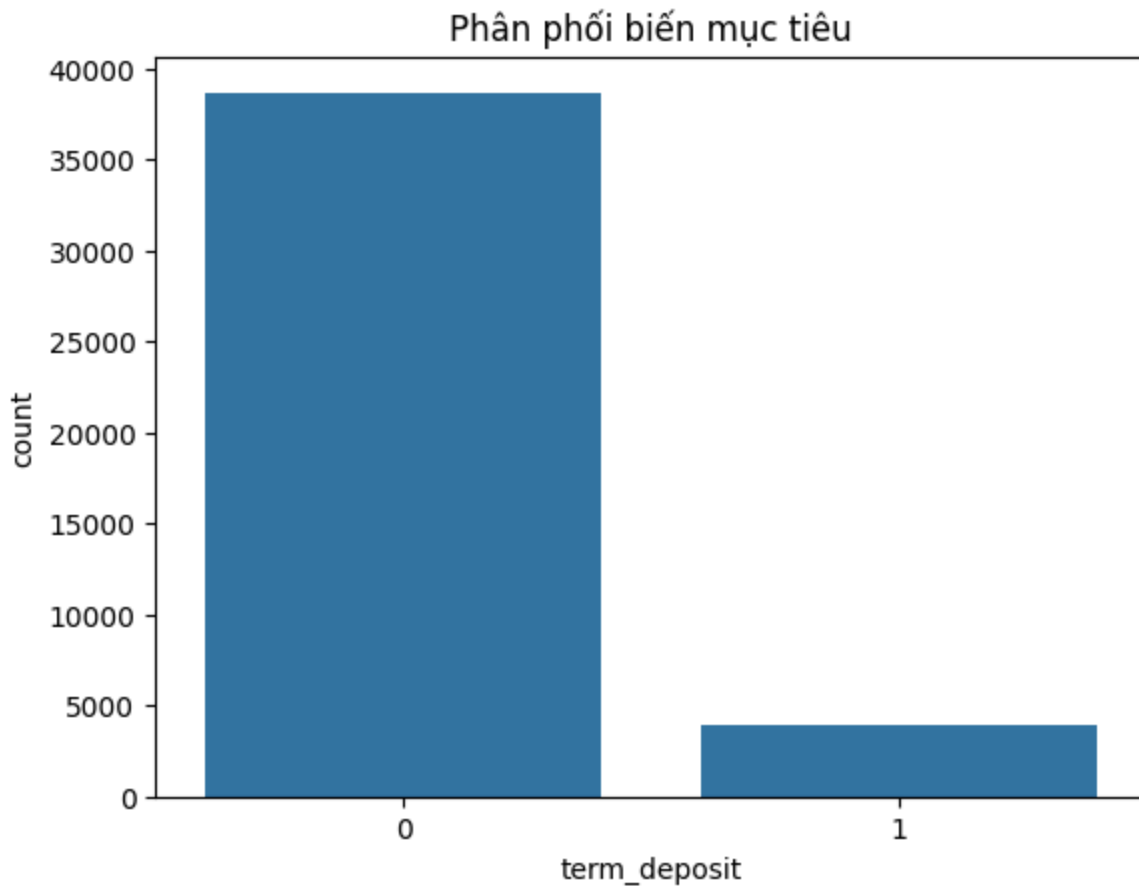
42639 rows × 15 columns



3.3.3.2. Phân tích khai phá dữ liệu

Trước hết có thể thấy rằng đây là một dữ liệu đã được làm sạch và không cần quá nhiều các bước tiền xử lý hay định dạng lại dữ liệu giống như ví dụ về giá nhà tại Hà Nội. Trước tiên, chúng ta xem phân phối của biến mục tiêu

```
sns.countplot(x='term_deposit', data=df)
plt.title('Phân phối biến mục tiêu')
plt.show()
df['term_deposit'].value_counts(normalize=True)
```



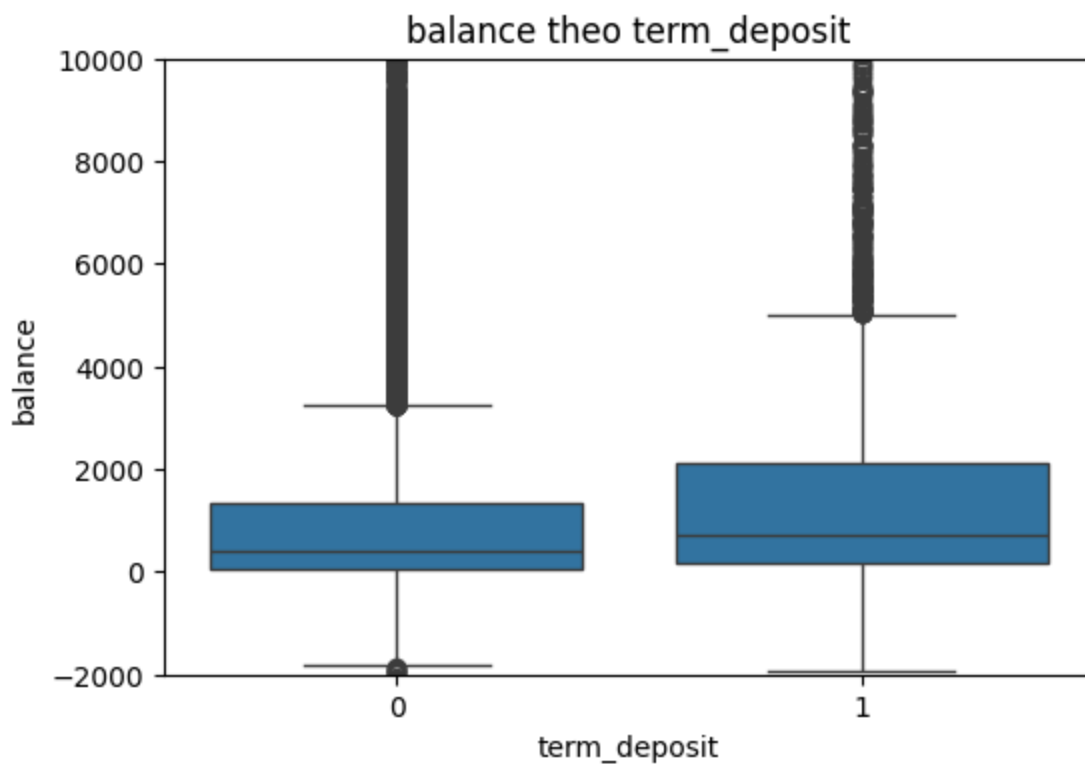
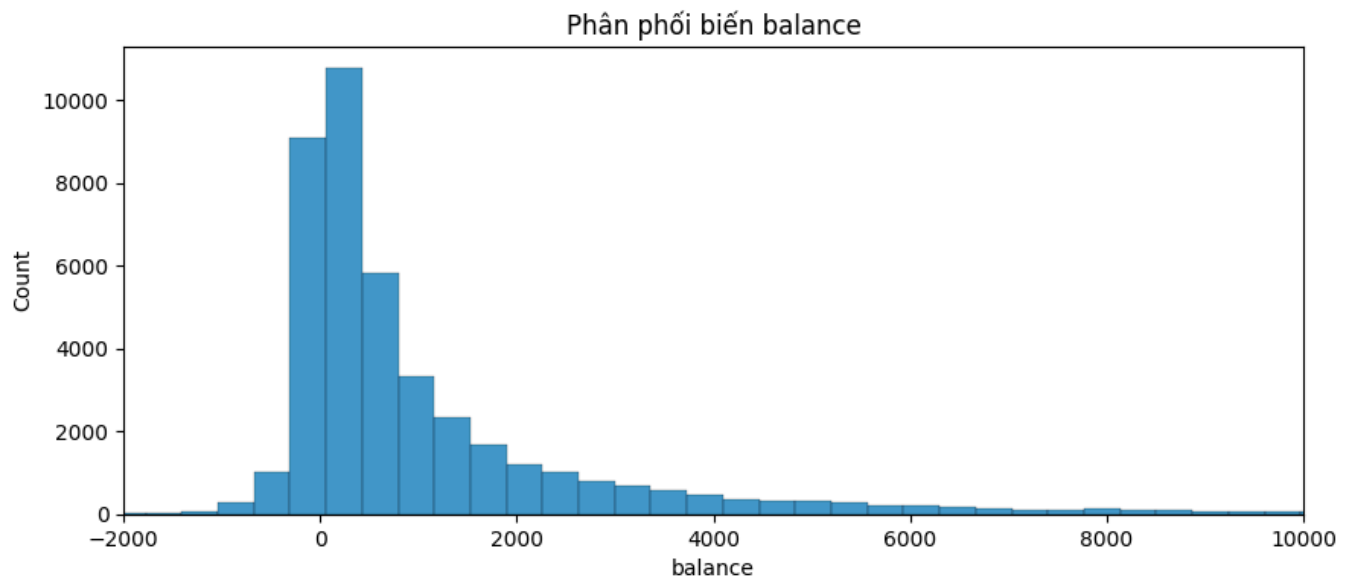
```
term_deposit
0    0.907104
1    0.092896
Name: proportion, dtype: float64
```

Có thể thấy rằng số tỷ lệ khách hàng đồng ý gửi tiết kiệm có kỳ hạn chỉ khoảng 9.3%, nghĩa là nếu chúng ta tiếp cận khách hàng một cách ngẫu nhiên mà không có tính toán, chỉ có 1 trên 10 khách hàng đồng ý gửi tiết kiệm. Bằng cách sử dụng mô hình học máy, chúng ta kỳ vọng rằng tỷ lệ khách hàng đồng ý sẽ lớn hơn so với lựa chọn ngẫu nhiên.

Chúng ta sẽ quan sát phân phối của một số biến liên tục quan trọng:

```
numerical_vars = ['balance']
for col in numerical_vars:
    plt.figure(figsize=(10, 4))
    sns.histplot(data=df, x=col, bins=300, color = "#0277B8")
    plt.title(f'Phân phối biến {col}')
    plt.xlim([-2000, 10000])
    plt.show()

    plt.figure(figsize=(6, 4))
    sns.boxplot(x='term_deposit', y=col, data=df)
    plt.title(f'{col} theo term_deposit')
    plt.ylim([-2000, 10000])
    plt.show()
```



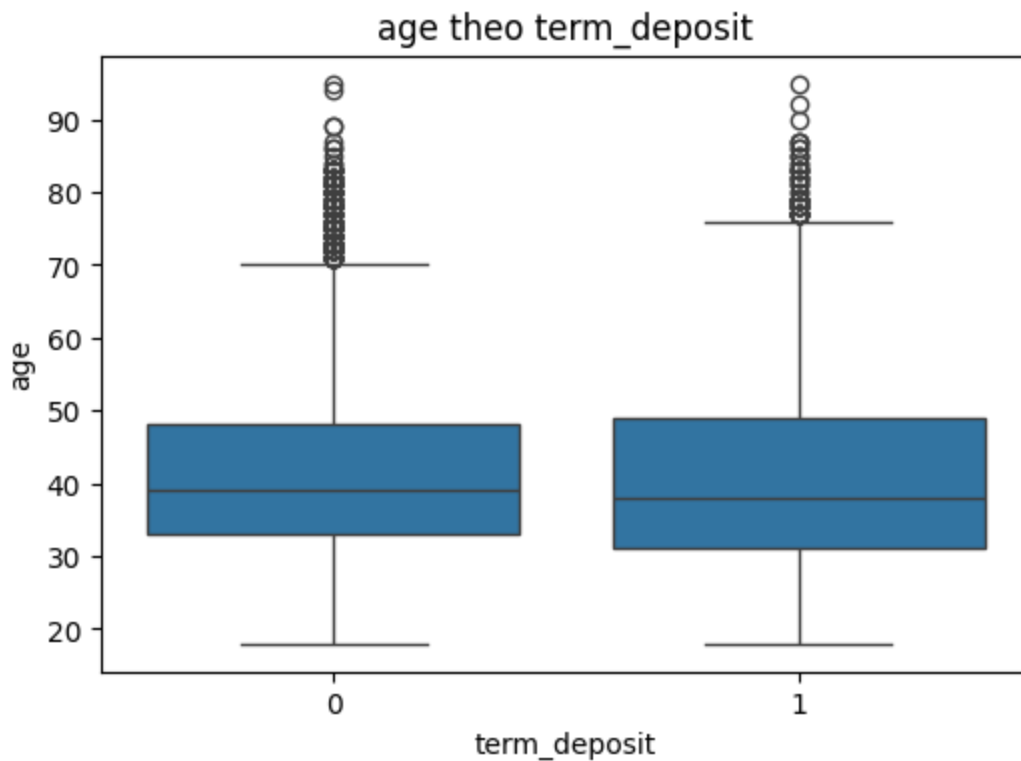
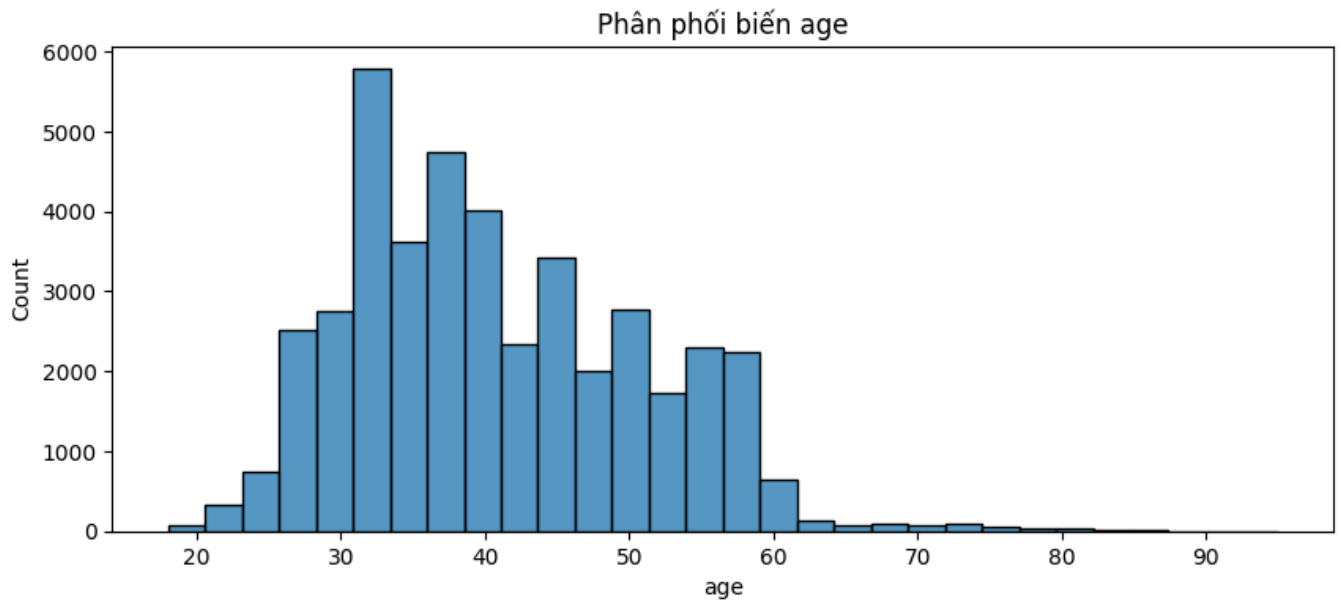
Phân tích thống kê mô tả đối với biến balance cho thấy phần lớn các quan sát tập trung trong khoảng giá trị từ -1000 đến 4000. Ngoài ra, có mối liên hệ dương giữa mức số dư tài khoản và khả năng khách hàng đồng ý tham gia tiền gửi có kỳ hạn, gợi ý rằng nhóm khách hàng có khả năng tài chính tốt hơn có xu hướng quan tâm nhiều hơn đến sản phẩm tiết kiệm của ngân hàng. Biến `balance` là một đặc trưng tốt để đưa vào trong mô hình học máy.

```

numerical_vars = ['age']
for col in numerical_vars:
    plt.figure(figsize=(10, 4))
    sns.histplot(data=df, x=col, bins=30)
    plt.title(f'Phân phối biến {col}')
    #plt.xlim([-2000, 10000])
    plt.show()

    plt.figure(figsize=(6, 4))
    sns.boxplot(x='term_deposit', y=col, data=df)
    plt.title(f'{col} theo term_deposit')
    #plt.ylim([-2000, 10000])
    plt.show()

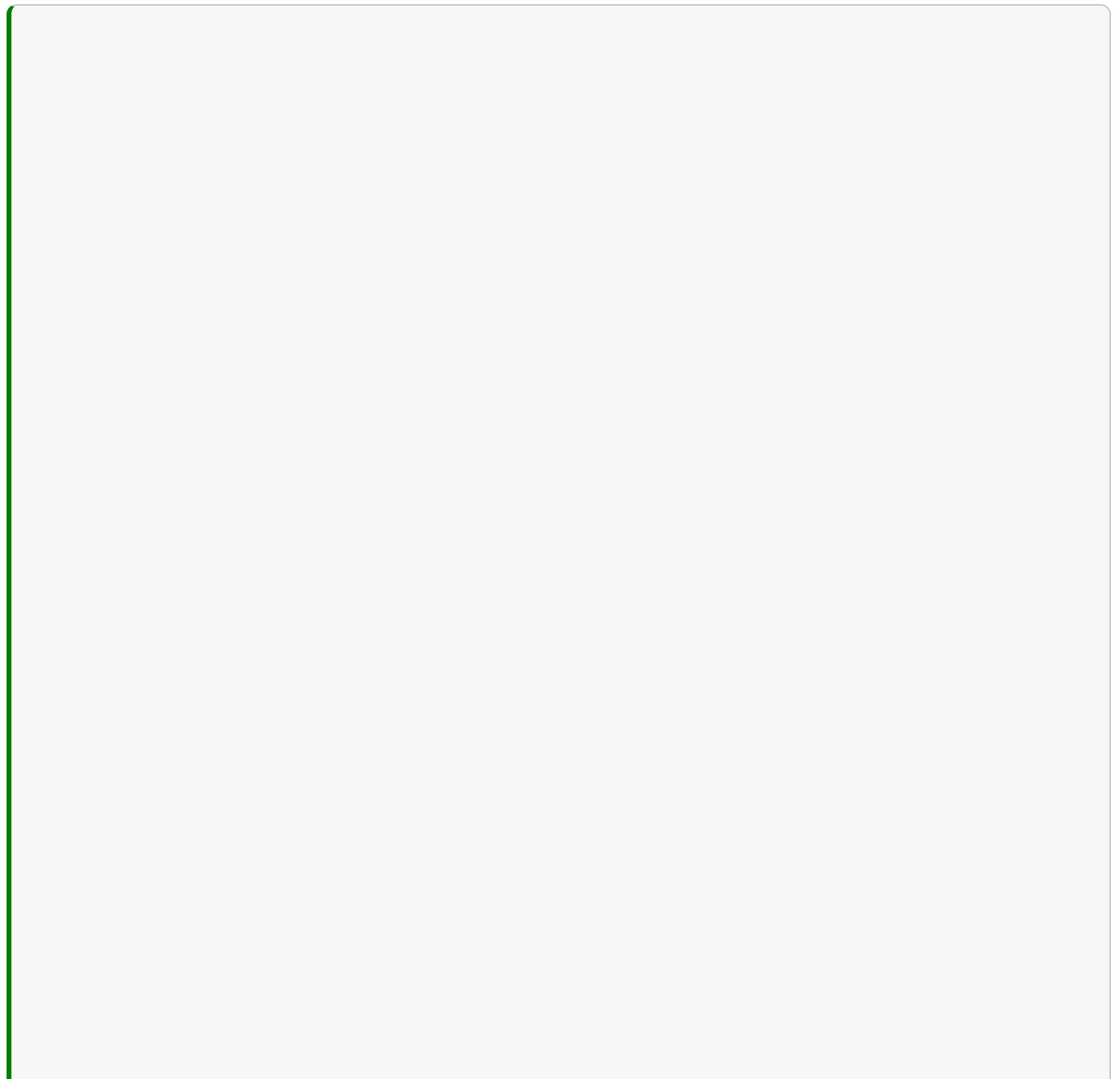
```



Khác với biến `balance`, `biến age` không thể hiện mối quan hệ tuyến tính hoặc xu hướng rõ ràng đối với xác suất khách hàng đồng ý gửi tiền tiết kiệm. Điều này cho thấy độ tuổi không phải là yếu tố quyết định hành vi tài chính liên quan đến sản phẩm tiền gửi có kỳ hạn trong tập khách hàng được khảo sát.

Mặc dù tuổi có thể ảnh hưởng đến các quyết định tài chính cá nhân, kết quả phân tích cho thấy không tồn tại mối quan hệ rõ ràng giữa độ tuổi và xu hướng gửi tiết kiệm. Điều này hàm ý rằng yếu tố độ tuổi có thể bị chi phối bởi các đặc điểm khác như nghề nghiệp, mức thu nhập, hoặc mục tiêu tài chính cá nhân, và do đó không đóng vai trò nổi bật khi xét độc lập trong dự đoán hành vi khách hàng.

Chúng ta sẽ đánh giá một số biến phân loại đến biến mục tiêu:



```

# Đếm số lượng theo housing/loan và term_deposit
housing_td = df.groupby(['housing', 'term_deposit']).size().unstack(fill_value=0)
loan_td = df.groupby(['loan', 'term_deposit']).size().unstack(fill_value=0)

# Tính tỷ lệ phần trăm
housing_td_pct = housing_td.div(housing_td.sum(axis=1), axis=0)
loan_td_pct = loan_td.div(loan_td.sum(axis=1), axis=0)

# Vẽ biểu đồ
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# BIỂU ĐỒ 1: Housing
axes[0].barh(housing_td.index.astype(str), housing_td[0], label='Không gửi', color='lightcoral')
axes[0].barh(housing_td.index.astype(str), -housing_td[1], label='Gửi tiết kiệm', color='steelblue')

axes[0].set_title("Phân bố term_deposit theo trạng thái Housing")
axes[0].set_ylabel("Housing (0 = Không vay, 1 = Có vay)")
axes[0].set_xlabel("Số lượng khách hàng")
axes[0].axvline(0, color='black', linewidth=0.8)
axes[0].legend()
axes[0].grid(axis='x', linestyle='--', alpha=0.6)

# Thêm nhãn %
for i, idx in enumerate(housing_td.index):
    l = housing_td.loc[idx, 0]
    r = housing_td.loc[idx, 1]
    axes[0].text(l + 1, i, f"{housing_td_pct.loc[idx, 0]*100:.1f}%", va='center', ha='left')

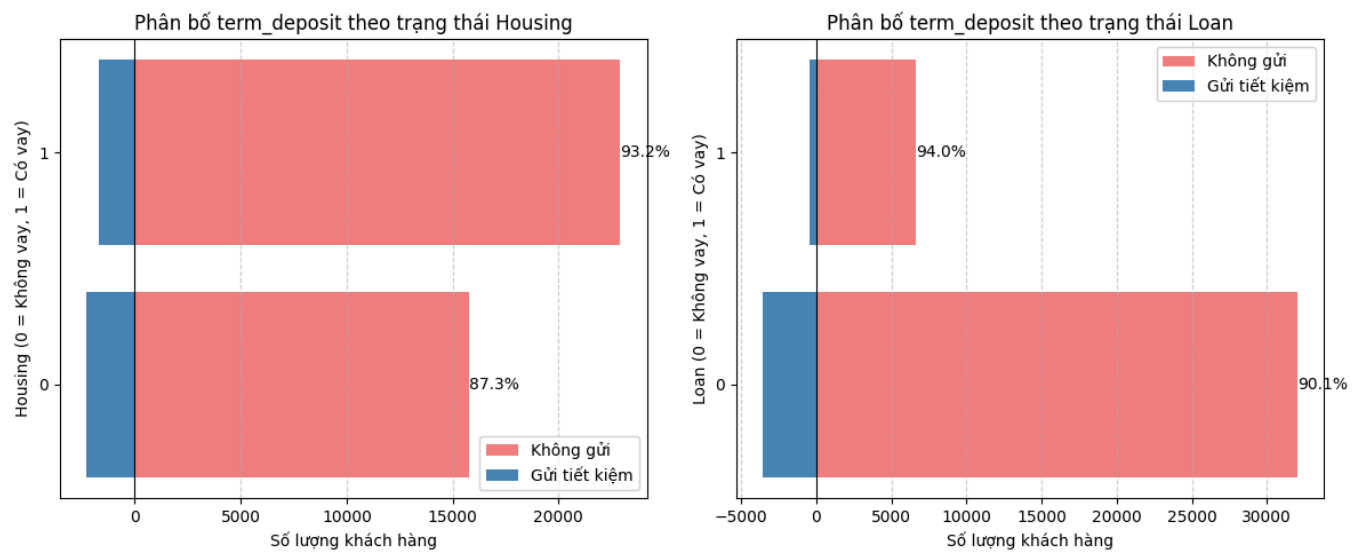
# BIỂU ĐỒ 2: Loan
axes[1].barh(loan_td.index.astype(str), loan_td[0], label='Không gửi', color='lightcoral')
axes[1].barh(loan_td.index.astype(str), -loan_td[1], label='Gửi tiết kiệm', color='steelblue')

axes[1].set_title("Phân bố term_deposit theo trạng thái Loan")
axes[1].set_ylabel("Loan (0 = Không vay, 1 = Có vay)")
axes[1].set_xlabel("Số lượng khách hàng")
axes[1].axvline(0, color='black', linewidth=0.8)
axes[1].legend()
axes[1].grid(axis='x', linestyle='--', alpha=0.6)

# Thêm nhãn %
for i, idx in enumerate(loan_td.index):
    l = loan_td.loc[idx, 0]
    r = loan_td.loc[idx, 1]
    axes[1].text(l + 1, i, f"{loan_td_pct.loc[idx, 0]*100:.1f}%", va='center', ha='left')

plt.tight_layout()
plt.show()

```

Phân tích dữ liệu cho thấy sự khác biệt rõ rệt trong hành vi gửi tiền tiết kiệm giữa các nhóm khách hàng có và không có khoản vay tài chính như vay mua nhà hoặc vay tiêu dùng. Cụ thể, khách hàng đang có các nghĩa vụ tài chính hiện hữu thường ít có xu hướng đăng ký sản phẩm tiền gửi có kỳ hạn. Điều này có thể được lý giải bởi áp lực chi trả định kỳ từ các khoản vay, làm giảm khả năng tích lũy vốn nhàn rỗi cũng như nhu cầu tiếp cận các sản phẩm tiết kiệm dài hạn.

```

# Tính số lượng khách hàng theo job và term_deposit
job_ct = df.groupby(['job', 'term_deposit']).size().unstack(fill_value=0)

# Tính tỷ lệ phần trăm đồng ý
job_pct = job_ct.div(job_ct.sum(axis=1), axis=0)

# Sắp xếp theo tổng số để biểu đồ trực quan hơn
job_ct = job_ct.loc[job_ct.sum(axis=1).sort_values(ascending=False).index]
job_pct = job_pct.loc[job_ct.index]

# Trục x
x = np.arange(len(job_ct))
width = 0.6

# Vẽ biểu đồ
fig, ax = plt.subplots(figsize=(14, 6))

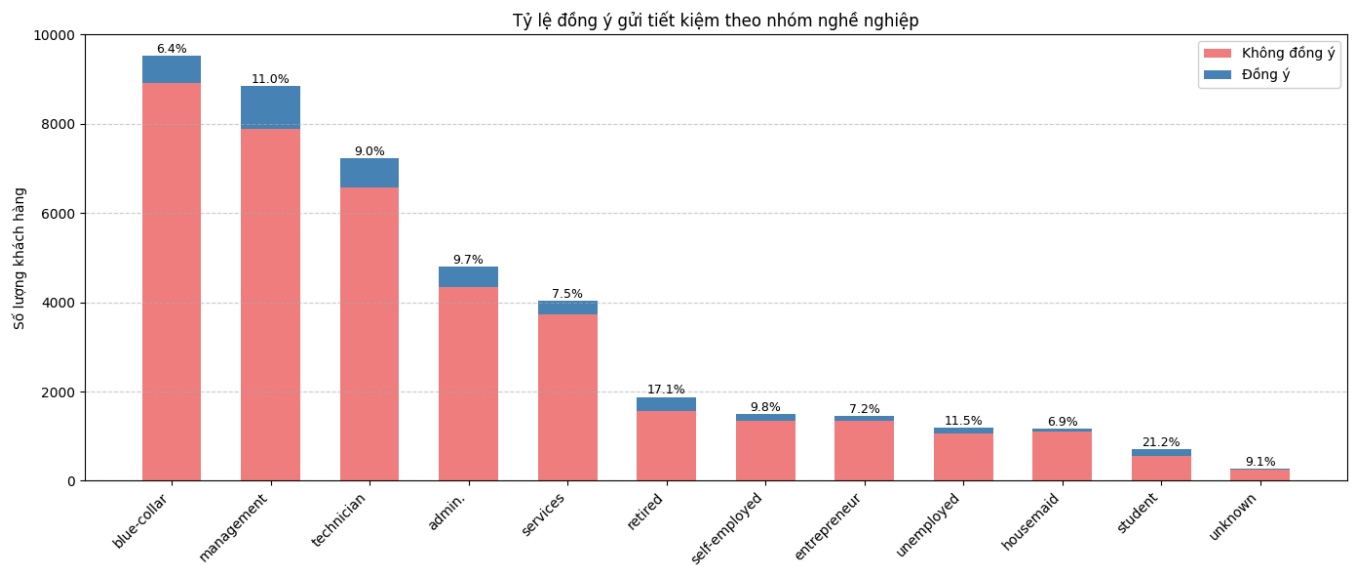
bars1 = ax.bar(x, job_ct[0], label='Không đồng ý', color='lightcoral', width=width)
bars2 = ax.bar(x, job_pct[1], bottom=job_ct[0], label='Đồng ý', color='steelblue', width=width)

# Cài đặt trục và tiêu đề
ax.set_xticks(x)
ax.set_xticklabels(job_ct.index, rotation=45, ha='right')
ax.set_ylabel("Số lượng khách hàng")
ax.set_title("Tỷ lệ đồng ý gửi tiết kiệm theo nhóm nghề nghiệp")
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.6)

# Thêm nhãn phần trăm ở phía trên cột
for i in range(len(x)):
    total = job_ct.iloc[i].sum()
    pct_yes = job_pct.iloc[i, 1] * 100
    top = job_ct.iloc[i, 0] + job_ct.iloc[i, 1]
    ax.text(x[i], top + 2, f"{pct_yes:.1f}%", ha='center', va='bottom', fontsize=9, color='steelblue')

plt.tight_layout()
plt.show()

```



Phân tích dữ liệu cho thấy nhóm khách hàng là sinh viên và người đã nghỉ hưu có xác suất đồng ý gửi tiền tiết kiệm cao hơn đáng kể so với các nhóm nghề nghiệp khác. Ngược lại, những người làm việc trong khối hành chính – văn phòng (ví dụ: nhóm 'admin.') lại thể hiện xu hướng ít quan tâm đến sản phẩm tiền gửi có kỳ hạn.

Có thể giải thích từ góc độ hành vi tài chính:

- Người nghỉ hưu thường có dòng tiền ổn định từ lương hưu và nhu cầu bảo toàn vốn cao. Do đó, họ có xu hướng lựa chọn các sản phẩm tài chính an toàn, như tiền gửi có kỳ hạn – nơi không yêu cầu mức độ rủi ro cao và vẫn đảm bảo sinh lời ổn định.
- Sinh viên, mặc dù có thu nhập hạn chế, lại có thể bị thu hút bởi các chương trình khuyến mãi hoặc chính sách ưu đãi từ ngân hàng. Đồng thời, một bộ phận sinh viên có thể đang quản lý khoản tiền được gia đình gửi định kỳ, từ đó dẫn đến hành vi gửi tiết kiệm để tích lũy cho tương lai gần.
- Nhân viên văn phòng có thể đang trong độ tuổi lao động tích cực, với nhiều lựa chọn đầu tư khác như chứng khoán, bảo hiểm, hoặc tiêu dùng lớn (mua nhà, mua xe), nên mức độ ưu tiên cho tiền gửi có kỳ hạn thấp hơn. Ngoài ra, nhóm này có thể có xu hướng thích các sản phẩm tài chính linh hoạt hơn so với tiền gửi cố định.

Quá trình phân tích khai phá dữ liệu sẽ tiếp tục được thực hiện đối với tất cả các biến được đưa vào mô hình. Đối với các biến phân loại có nhiều mức giá trị (category levels), bạn đọc cần lưu ý rằng nguyên tắc chuyển đổi các biến này thành đặc trưng đầu vào có ý nghĩa là dựa trên việc nhóm các mức có giá trị trung bình của biến mục tiêu tương đồng lại với nhau. Cách tiếp cận này không chỉ giúp giảm số lượng đặc trưng cần xử lý mà còn đảm bảo tính ổn định và khả năng giải thích của mô hình.

3.3.3.3. Xây dựng mô hình học máy

Sau quá trình phân tích khai phá, chúng ta sẽ lựa chọn các biến phân loại bao gồm `job`, `marital`, `education`, `housing`, `loan`, `previous`, và `campaign` vào mô hình dự đoán. Biến `balance` là biến liên tục duy nhất được sử dụng.

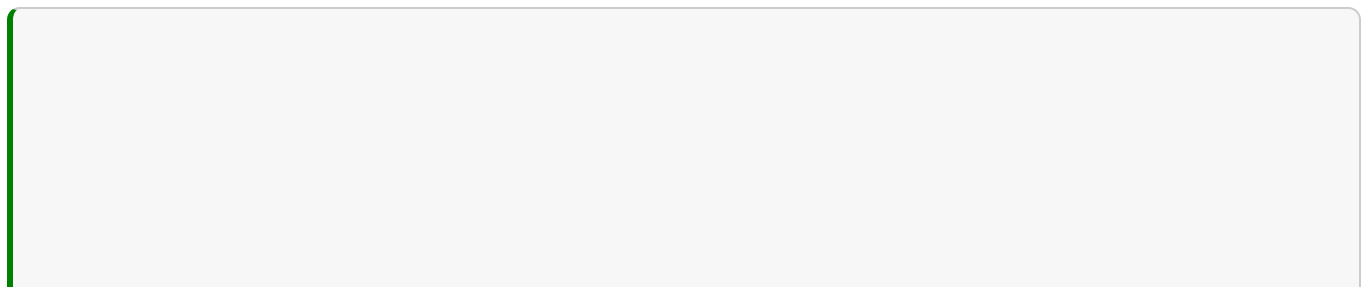
```
path = 'data/BankCustomerData_clean.csv'
df = pd.read_csv(path, index_col=0, parse_dates=True)
df
```

	ID	age	job	marital	education	default	balance	housing	l
1	1	58	management	2	tertiary	0	2143	1	
2	2	44	technician	1	secondary	0	29	1	
3	3	33	entrepreneur	2	secondary	0	2	1	
4	4	47	blue-collar	2	unknown	0	1506	1	
5	5	33	unknown	1	unknown	0	1	0	
...	...	...	...	...	...	...	...	...	
42635	42635	21	student	1	secondary	0	2488	0	
42636	42636	87	retired	2	primary	0	2190	0	
42637	42637	34	blue-collar	2	primary	0	6718	0	
42638	42638	22	student	1	secondary	0	254	0	
42639	42639	32	management	1	tertiary	0	1962	0	

42639 rows × 16 columns



Chúng ta chia dữ liệu thành phần huấn luyện mô hình và kiểm tra mô hình theo tỷ lệ 50%-50%



```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    df[["job", "marital", "education", "housing", "loan", "previous", "campaign", "balance"],
    random_state = 0, test_size=0.5)
```

X_train

	job	marital	education	housing	loan	previous	campaign	balance
31256	retired	1	unknown	0	1	0	3	1503
19548	technician	2	secondary	0	0	0	2	1203
32854	management	1	tertiary	1	0	3	1	2143
40921	blue-collar	1	primary	0	0	0	1	1203
2001	management	2	primary	1	0	0	4	1503
...	...	...	...	...	...	...	...	...
32104	blue-collar	2	secondary	1	0	0	4	1203
30404	management	1	tertiary	0	0	0	1	2143
21244	management	2	tertiary	1	0	0	5+	2143
42614	management	2	tertiary	0	0	0	2	2143
2733	blue-collar	1	secondary	1	0	0	4	1203

21319 rows × 8 columns

Tương tự như ví dụ trước, chúng ta biến đổi biến phân loại thành biến dummy:

```
# Chuyển đổi biến sang kiểu one hot
X_train_encoded = pd.get_dummies(X_train, drop_first=True)
X_test_encoded = pd.get_dummies(X_test, drop_first=True)

# Đồng bộ hóa cột giữa train/test
X_train_encoded, X_test_encoded = X_train_encoded.align(X_test_encoded, join='left', axis=1)
```

Mô hình hồi quy logistic là một trong những phương pháp kinh điển thường được sử dụng khi biến mục tiêu là biến nhị phân. Mặc dù tồn tại một số vấn đề cần được xem xét kỹ lưỡng trước khi xây dựng mô hình – chẳng hạn như sự mất cân bằng giữa hai lớp của biến mục tiêu – trong phạm vi hiện tại, chúng ta tạm thời chưa đi sâu vào các khía cạnh này và sẽ trực tiếp thực thi mô hình. Những vấn đề liên quan đến chất lượng dữ liệu, đặc biệt là mất cân bằng lớp, sẽ được phân tích và thảo luận chi tiết hơn trong các phần tiếp theo của cuốn sách.

```
from sklearn.linear_model import LogisticRegression

# Huấn luyện mô hình Logistic Regression
model = LogisticRegression(max_iter=1000, class_weight='balanced')
model.fit(X_train_encoded, Y_train)

# Dự đoán
Y_pred = model.predict(X_test_encoded)
```

Khi xây dựng mô hình dự đoán với biến mục tiêu nhị phân, việc đánh giá hiệu quả mô hình thường được thực hiện thông qua Confusion Matrix. Đây là một bảng 2x2 thể hiện số lượng dự đoán đúng và sai của mô hình, phân theo từng lớp của biến mục tiêu. Đối với mô hình phân loại nhị phân, confusion matrix bao gồm bốn thành phần chính:

	Dự đoán: 0	Dự đoán: 1
Thực tế: 0	TN	FP
Thực tế: 1	FN	TP

- **True Positive (TP):**

Số lượng quan sát có giá trị thực là 1 và được mô hình dự đoán đúng là 1.

→ Dự đoán đúng trường hợp dương tính.

- **True Negative (TN):**

Số lượng quan sát có giá trị thực là 0 và được dự đoán đúng là 0.

→ Dự đoán đúng trường hợp âm tính.

- **False Positive (FP):**

Số lượng quan sát có giá trị thực là 0 nhưng bị mô hình dự đoán nhầm là 1.

→ Dự đoán sai dương tính (hay còn gọi là *Type I error*).

- **False Negative (FN):**

Số lượng quan sát có giá trị thực là 1 nhưng bị mô hình dự đoán nhầm là 0.

→ Dự đoán sai âm tính (*Type II error*).

Từ các thành phần trên, ta có thể tính được nhiều **chỉ số đánh giá quan trọng**, bao gồm:

- **Accuracy** (Độ chính xác tổng thể):
[$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$]
- **Precision** (Độ chính xác cho lớp dương):
[$\text{Precision} = \frac{TP}{TP + FP}$]
- **Recall** (Khả năng phát hiện lớp dương):
[$\text{Recall} = \frac{TP}{TP + FN}$]
- **F1-score** (Trung bình điều hòa của Precision và Recall):
[$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$]

Confusion Matrix cung cấp một cái nhìn chi tiết và cân bằng hơn về hiệu quả của mô hình phân loại so với chỉ số đơn lẻ như accuracy. Đặc biệt trong các bài toán mất cân bằng lớp, các chỉ số như precision, recall và F1-score thường phản ánh chính xác hơn năng lực của mô hình đối với lớp mục tiêu quan trọng.

Confusion matrix được tính toán cho kết quả của mô hình hồi quy logistic như sau:

```
from sklearn.metrics import classification_report, confusion_matrix

# Đánh giá mô hình
print("Confusion Matrix:\n", confusion_matrix(Y_test, Y_pred))
print("\nClassification Report:\n", classification_report(Y_test, Y_pred))
```

Confusion Matrix:

```
[[12381  6955]
 [   802 1182]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.64	0.76	19336
1	0.15	0.60	0.23	1984
accuracy			0.64	21320
macro avg	0.54	0.62	0.50	21320
weighted avg	0.87	0.64	0.71	21320

Có thể nhận xét kết quả như sau:

- Recall của lớp 1 đạt 60%: cho thấy mô hình đã có khả năng phát hiện đáng kể số lượng khách hàng thực sự gửi tiền – đây là yếu tố quan trọng trong bài toán marketing nhằm mục tiêu.
- F1-score của lớp 0 là 0.76, chứng tỏ mô hình vẫn duy trì được hiệu suất khá tốt với nhóm khách hàng không gửi tiền.
- Precision của lớp 1 rất thấp (0.15): chỉ 15% trong số những khách hàng được dự đoán sẽ gửi tiền là đúng thực tế.
- F1-score của lớp 1 chỉ đạt 0.23, phản ánh độ chính xác tổng thể đối với nhóm mục tiêu còn yếu.

Mặc dù mô hình vẫn còn nhiều điểm cần cải thiện, tuy nhiên chúng ta vẫn có thể thấy được hiệu quả của mô hình trong lựa chọn khách hàng tiềm năng. Quay trở lại với mục tiêu ban đầu, chúng ta cần lựa chọn 2000 khách hàng có khả năng cao chấp nhận mở sổ tiết kiệm. Nếu chúng ta lựa chọn ngẫu nhiên, số lượng khách hàng đồng ý rơi vào khoảng 9.3%, tương đương với khoảng 185 khách hàng.

Nếu sử dụng kết quả của mô hình logistic, chúng ta sẽ lựa chọn 2000 khách hàng có xác suất đồng ý mở sổ tiết kiệm cao nhất trong hơn 20 nghìn khách hàng của tập dữ liệu.

```
# Dự đoán xác suất đồng ý gửi tiết kiệm
y_proba = model.predict_proba(X_test_encoded)[: , 1]

# Đưa xác suất vào DataFrame
df_proba = X_test_encoded.copy()
df_proba['probability'] = y_proba

# Thêm biến nhãn thật
df_proba['true_label'] = Y_test.values

# Sắp xếp theo xác suất giảm dần
df_top2000 = df_proba.sort_values(by='probability', ascending=False).head(2000)

# Số lượng khách hàng đồng ý
sum(df_top2000['true_label'])
```

494

Dựa trên kết quả dự đoán xác suất từ mô hình hồi quy logistic, khi lựa chọn 2.000 khách hàng có xác suất đồng ý cao nhất, có tổng cộng 494 khách hàng thực sự đồng ý mở sổ tiết kiệm. Điều này tương đương với tỷ lệ chuyển đổi là 24.7%.

So sánh với tỷ lệ đồng ý trung bình trong toàn bộ tập dữ liệu là 9.3%, tương ứng với lựa chọn ngẫu nhiên, mô hình đã giúp tăng gần gấp 3 lần tỷ lệ chuyển đổi, cho thấy hiệu quả đáng kể trong việc ưu tiên tiếp cận nhóm khách hàng tiềm năng.

Chúng ta cũng có thể sử dụng mô hình *rừng ngẫu nhiên* như trong ví dụ kể trên để dự đoán khách hàng có hay không đồng ý mở thẻ tín dụng. Mô hình được thực thi và cho kết quả như sau:

```
from sklearn.ensemble import RandomForestRegressor

# Khởi tạo mô hình Random Forest
rf_model = RandomForestRegressor(
    n_estimators = 500,      # số lượng cây
    max_depth = 10,         # độ sâu tối đa
    max_features = 5,       # số lượng đặc trưng của mỗi cây quyết định
    random_state = 10,      # khởi tạo ngẫu nhiên để tái lập kết quả
)

# Huấn luyện mô hình với dữ liệu huấn luyện
rf_model.fit(X_train_encoded, Y_train)
Y_pred = rf_model.predict(X_test_encoded)

df_proba = X_test_encoded.copy()
df_proba['probability'] = Y_pred

# Thêm biến nhãn thật
df_proba['true_label'] = Y_test.values

# Sắp xếp theo xác suất giảm dần
df_top2000 = df_proba.sort_values(by='probability', ascending=False).head(2000)

# Số lượng khách hàng đồng ý
sum(df_top2000['true_label'])
```

511

Kết quả phân tích cho thấy số lượng khách hàng thực sự đồng ý mở sổ tiết kiệm trong nhóm được mô hình lựa chọn đã tăng từ 494 lên 511 khi chuyển từ mô hình hồi quy logistic sang mô hình rừng ngẫu nhiên. Điều này cho thấy hiệu suất dự báo của mô hình rừng ngẫu nhiên có phần vượt trội hơn so với mô hình hồi quy logistic truyền thống trong trường hợp cụ thể này.

Tuy nhiên, cần lưu ý rằng hồi quy logistic là một mô hình dạng tham số, cho phép diễn giải trực tiếp mối quan hệ giữa các biến đầu vào và biến mục tiêu. Ngược lại, mô hình rừng ngẫu nhiên tuy có khả năng dự báo mạnh mẽ hơn, lại thuộc nhóm mô hình phi tham số, do đó khó đưa ra

các suy diễn định lượng rõ ràng. Đây chính là ví dụ điển hình cho một vấn đề phổ biến trong học máy: sự đánh đổi giữa khả năng dự báo và khả năng giải thích của mô hình.

Từ góc độ ứng dụng, việc sử dụng các mô hình dự đoán cho phép tổ chức không chỉ tối ưu hóa chi phí tiếp thị thông qua việc xác định nhóm khách hàng tiềm năng, mà còn gia tăng đáng kể tỷ lệ chuyển đổi trong các chiến dịch. Kết quả này là một minh chứng cụ thể cho tính khả thi và hiệu quả của việc tích hợp các mô hình học máy vào chiến lược chăm sóc khách hàng và phân bổ nguồn lực trong lĩnh vực ngân hàng.

